



**Faculty of Engineering and Technology**  
**Electrical and Computer Engineering Department**

**Computer Networks**  
**ENCS3320**  
**Project #1**

| <b>Name</b>      | <b>ID</b> | <b>Section</b> |
|------------------|-----------|----------------|
| Sari Abdal-ghani | 1220982   | 2              |
| Mohammad Zaid    | 1221265   | 1              |
| Mohammad Omar    | 1221332   | 1              |

**Submission Date: 1/12/2024**

# Table of Contents

|                                                                                 |    |
|---------------------------------------------------------------------------------|----|
| <b>Table of Contents</b> .....                                                  | 1  |
| <b>Table of Figures</b> .....                                                   | 2  |
| <b>Theory</b> .....                                                             | 4  |
| <b>1. Transport Layer Protocols:</b> .....                                      | 4  |
| <b>2. TCP:</b> .....                                                            | 5  |
| <b>3. UDP:</b> .....                                                            | 5  |
| <b>4. Client – Server Model:</b> .....                                          | 6  |
| <b>Procedure</b> .....                                                          | 7  |
| <b>1. Task 1– Network Commands and Wireshark:</b> .....                         | 7  |
| <b>1.1 Commands:</b> .....                                                      | 7  |
| <b>1.2 Perform Actions:</b> .....                                               | 8  |
| <b>1.3 Wireshark packet analyzer:</b> .....                                     | 14 |
| <b>2. Task 2-Web Server:</b> .....                                              | 15 |
| <b>2.1 Socket Setup:</b> .....                                                  | 15 |
| <b>2.2 Request Handling:</b> .....                                              | 16 |
| <b>2.3 File Serving:</b> .....                                                  | 17 |
| <b>2.4 HTML/CSS Part:</b> .....                                                 | 19 |
| <b>2.5 Run the Cases:</b> .....                                                 | 24 |
| <b>3. Task 3 – UDP Client-Server Trivia Game Using Socket Programming</b> ..... | 33 |
| <b>3.1 What is the Task?</b> .....                                              | 33 |
| <b>3.2 Run the UDP server</b> .....                                             | 33 |
| <b>3.3 Join Player’s ( Client’s ):</b> .....                                    | 36 |
| <b>3.4 Start the Game:</b> .....                                                | 39 |
| <b>3.5 using another device:</b> .....                                          | 50 |
| <b>Alternative Solutions, Issues, and Limitations:</b> .....                    | 52 |
| <b>References</b> .....                                                         | 53 |
| <b>Teamwork</b> .....                                                           | 54 |

## Table of Figures

|                                                                       |    |
|-----------------------------------------------------------------------|----|
| Figure 1: Transport Layer in Layers (source: omniSecu.com) .....      | 4  |
| Figure 2: The Difference between UDP and TCP .....                    | 5  |
| Figure 3: Client-Server Network (source: zenaromor.com) .....         | 6  |
| Figure 4: ipconfig/All command .....                                  | 8  |
| Figure 5: ping to other device .....                                  | 9  |
| Figure 6: ping to website.....                                        | 10 |
| Figure 7: Tracert command .....                                       | 11 |
| Figure 8: nslookup command.....                                       | 12 |
| Figure 9: telnet command .....                                        | 13 |
| Figure 10: Try wirshark .....                                         | 14 |
| Figure 11: code for socket setup .....                                | 15 |
| Figure 12: code for Request Handling.....                             | 16 |
| Figure 13: code for html and css en page loading.....                 | 17 |
| Figure 14: code for send all page images.....                         | 18 |
| Figure 15: html code for member section .....                         | 19 |
| Figure 16: html code for the prvided links.....                       | 20 |
| Figure 17: html code for topic.....                                   | 20 |
| Figure 18: supporting page .....                                      | 21 |
| Figure 19: html code for supporting page .....                        | 21 |
| Figure 20: code for find case.....                                    | 22 |
| Figure 21: code for not found image case.....                         | 22 |
| Figure 22: code for not found video case.....                         | 23 |
| Figure 23: Run the web server.....                                    | 24 |
| Figure 24: handle the page request .....                              | 25 |
| Figure 25: open en website .....                                      | 25 |
| Figure 26: open ar page.....                                          | 26 |
| Figure 27: http request output on the terminal 1 .....                | 26 |
| Figure 28: http request output on the terminal 2 .....                | 27 |
| Figure 29: css request.....                                           | 27 |
| Figure 30: open the website on other devise.....                      | 28 |
| Figure 31: test for founded image .....                               | 29 |
| Figure 32: output for founded image .....                             | 29 |
| Figure 33: test for not founded image .....                           | 30 |
| Figure 34: output for not founded image .....                         | 30 |
| Figure 35: video directory in our project file.....                   | 31 |
| Figure 36: output for founded video .....                             | 31 |
| Figure 37:output for not founded video .....                          | 32 |
| Figure 38: write the command the run the server and the clients ..... | 33 |
| Figure 39: code of initialize run of UDP server.....                  | 34 |
| Figure 40: first message from server when run.....                    | 34 |
| Figure 41: code for start run UDP server.....                         | 35 |
| Figure 42: code for time out without response .....                   | 35 |

|                                                                               |    |
|-------------------------------------------------------------------------------|----|
| Figure 43: code for server to read the questions .....                        | 35 |
| Figure 44: code for server to choose random set of question .....             | 36 |
| Figure 45: when we want to join player .....                                  | 36 |
| Figure 46: The sever announce the player join and wait .....                  | 37 |
| Figure 47: code for client to ask the player about IP,port and his name ..... | 37 |
| Figure 48: receive message from sever .....                                   | 38 |
| Figure 49:code for server handle request to join and send message .....       | 38 |
| Figure 50: 2 players join .....                                               | 39 |
| Figure 51: server code when the number of player become 2 .....               | 39 |
| Figure 52: client receive the start message .....                             | 40 |
| Figure 53: Server send the question and wait the answer .....                 | 40 |
| Figure 54: code for Server to send the question to all players .....          | 41 |
| Figure 55: the players answer first question .....                            | 41 |
| Figure 56: server send the score of the players .....                         | 42 |
| Figure 57: 3 players with right answers .....                                 | 43 |
| Figure 58: code for Receive the question and answer it .....                  | 44 |
| Figure 59: receive the answer .....                                           | 44 |
| Figure 60: check player answer .....                                          | 45 |
| Figure 61: duplicate answer .....                                             | 45 |
| Figure 62: No point for wrong answer.....                                     | 46 |
| Figure 63: winner of the round .....                                          | 46 |
| Figure 64: server print the winner .....                                      | 47 |
| Figure 65: player join after game start .....                                 | 47 |
| Figure 66: remover player from the game .....                                 | 48 |
| Figure 67: exit the game.....                                                 | 49 |
| Figure 68: client send exit to server .....                                   | 49 |
| Figure 69: server receive exit message and remove him .....                   | 49 |
| Figure 70: another device join to the game .....                              | 50 |
| Figure 71: Other device play the game .....                                   | 51 |

# Theory

## 1. Transport Layer Protocols:

“Transport Layer Protocols, TCP and UDP, are essential for data exchange in networking. TCP guarantees reliable data transmission through retransmission and a three-way handshake, while UDP prioritizes fast data transfer.

Transport layer protocols are crucial components of networking because they provide a way to transmit data packets from one network node to another. Transport protocols serve various purposes, such as streaming video, navigating the internet, and executing transactions. The two core protocols are TCP and UDP.”[1]

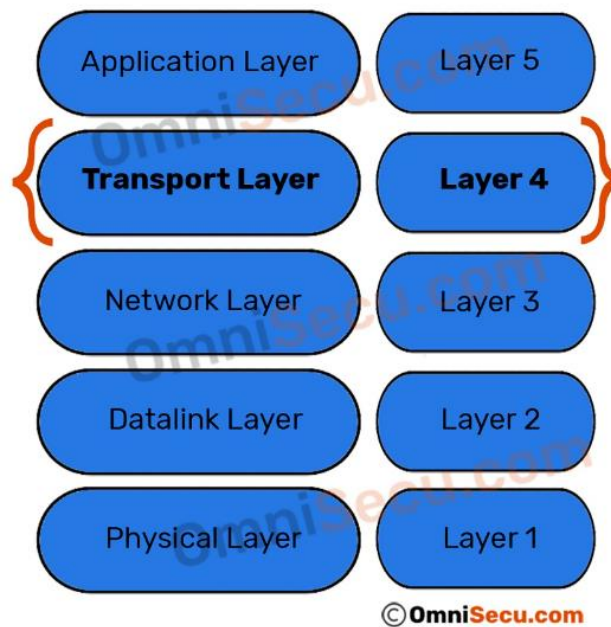


Figure 1: Transport Layer in Layers (source: omniSecu.com)

## 2. TCP:

“TCP stands for **Transmission Control Protocol**. It is a transport layer protocol that facilitates the transmission of packets from source to destination. It is a connection-oriented protocol that means it establishes the connection prior to the communication that occurs between the computing devices in a network. This protocol is used with an **IP** protocol, so together, they are referred to as a **TCP/IP**.”

The main functionality of the TCP is to take the data from the application layer. Then it divides the data into a several packets, provides numbering to these packets, and finally transmits these packets to the destination. The TCP, on the other side, will reassemble the packets and transmits them to the application layer. As we know that TCP is a connection-oriented protocol, so the connection will remain established until the communication is not completed between the sender and the receiver.”[2]

## 3. UDP:

“User Datagram Protocol (UDP) is a Transport Layer protocol. UDP is a part of the Internet Protocol suite, referred to as UDP/IP suite. Unlike TCP, it is an unreliable and connectionless protocol. So, there is no need to establish a connection before data transfer. The UDP helps to establish low-latency and loss-tolerating connections over the network. The UDP enables process-to-process communication.”[3]

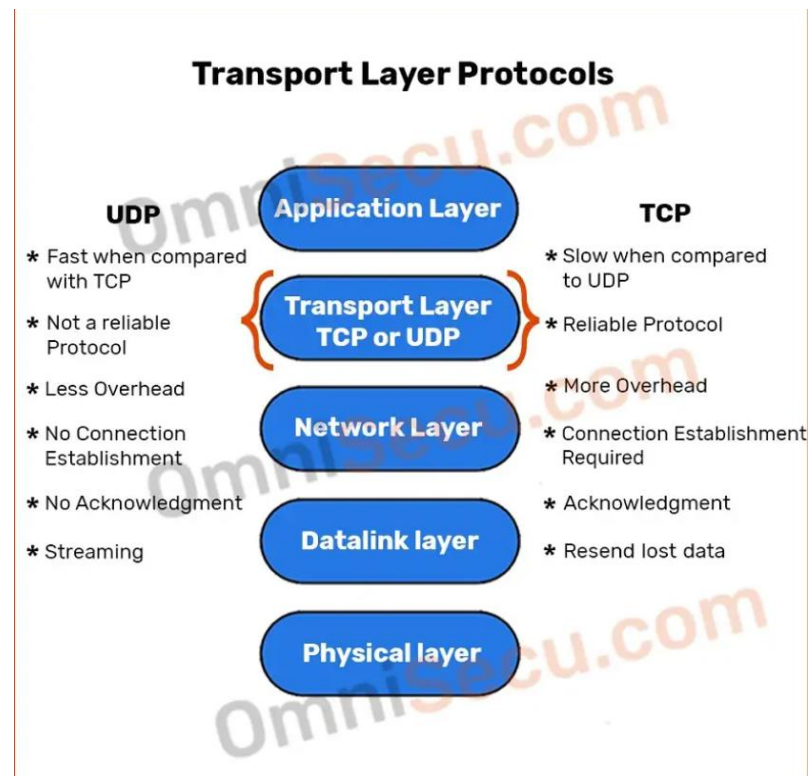


Figure 2: The Difference between UDP and TCP

## 4. Client – Server Model:

“The client-server model describes how a server gives one or more clients access to resources and services. Mail servers, web servers, and file servers are examples of servers. Client devices, including desktops, laptops, tablets, and mobile devices, have access to the resources on each of these servers. Clients and servers often have a one-to-many connection, which means that a single server can supply resources to several clients at the same time.

When a client requests a connection with a server, the server has the option of accepting or rejecting the request. If the connection is accepted, the server builds and maintains a protocol-specific connection with the client. To send a message, for instance, an email client may demand an SMTP connection to a mail server. The mail server's SMTP program will then request authentication information, such as the email address and password. The server will deliver the email to the designated recipient if the credentials match an account on the mail server.”[4]

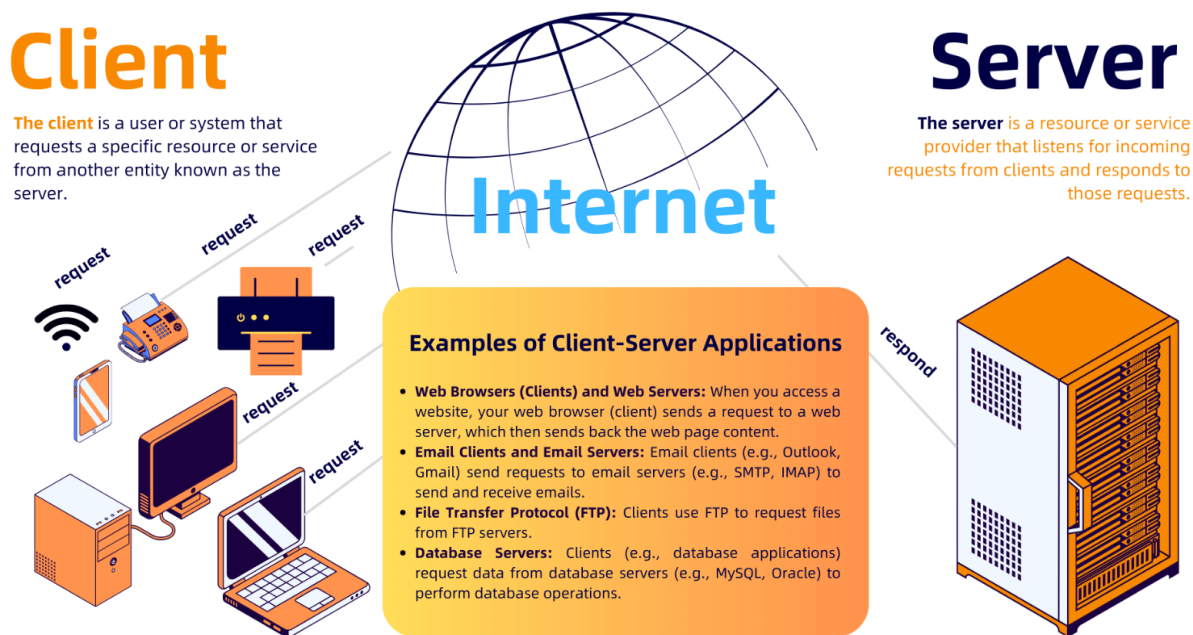


Figure 3: Client-Server Network (source: zenaromor.com)

## Procedure

### 1. Task 1– Network Commands and Wireshark:

#### 1.1 Commands:

**1. Ipconfig:** used to display or manage the network configuration of a computer. It provides details such as the IP address, subnet mask, and default gateway of network interfaces and can also refresh and DNS settings.

**2. Ping:** command that sends an Internet Control Message Protocol (ICMP) Echo Request message to a specified remote computer to verify IP-level connectivity.

**3. Tracert:** command that sends Internet Control Message Protocol (ICMP) Echo Request messages to a specified remote computer with increasing Time to Live (TTL) field values and displays the IP address and hostname, if available, of the router interfaces between the source and destination.

**4. Nslookup:** command most commonly used to display the hostname of an entered IP address. by querying your configured DNS server to discover the IP address.

**5. Telnet:** The telnet command is used to communicate with remote computers that use the Telnet protocol.



## 1.2 Perform Actions:

- **ipconfig /all :**

for this command I will know some information like:

IP address: 192.168.0.189

Subnet mask: 255.255.255.0

Default gateway: 192.168.0.1

DNS: 8.8.8.8

```
Command Prompt
Wireless LAN adapter Wi-Fi:
Connection-specific DNS Suffix . : www.tendawifi.com
Description . . . . . : Intel(R) Wi-Fi 6 AX201 160MHz
Physical Address. . . . . : BE-16-80-E9-55-34
DHCP Enabled. . . . . : Yes
Autoconfiguration Enabled . . . . : Yes
Link-local IPv6 Address . . . . . : fe80::c2df:f608:de0:931%19(Preferred)
IPv4 Address. . . . . : 192.168.0.189(Preferred)
Subnet Mask . . . . . : 255.255.255.0
Lease Obtained. . . . . : Saturday, November 30, 2024 12:42:07 AM
Lease Expires . . . . . : Sunday, December 1, 2024 3:07:03 PM
Default Gateway . . . . . : 192.168.0.1
DHCP Server . . . . . : 192.168.0.1
DHCPv6 IAID . . . . . : 331224704
DHCPv6 Client DUID. . . . . : 00-03-00-01-8E-16-80-E9-55-34
DNS Servers . . . . . : 8.8.8.8
                        8.8.4.4
NetBIOS over Tcpip. . . . . : Enabled

Ethernet adapter Bluetooth Network Connection:

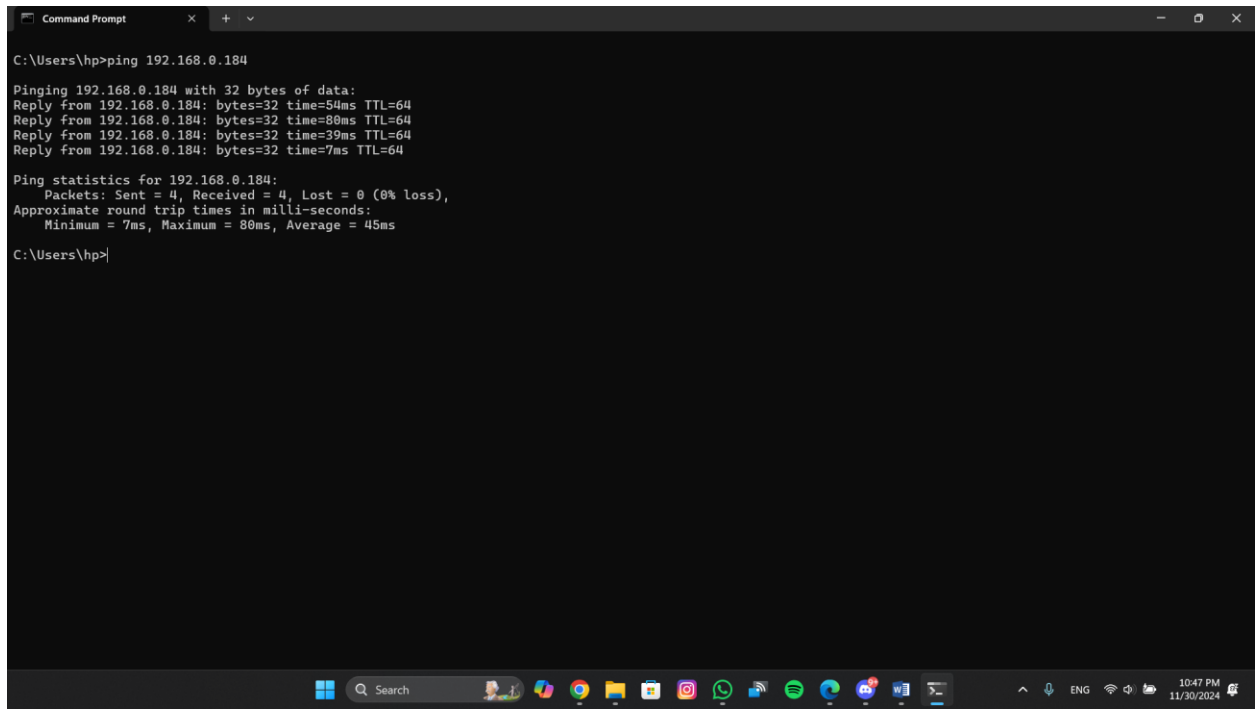
Media State . . . . . : Media disconnected
Connection-specific DNS Suffix . :
Description . . . . . : Bluetooth Device (Personal Area Network)
Physical Address. . . . . : A0-59-50-2E-56-FC
DHCP Enabled. . . . . : Yes
Autoconfiguration Enabled . . . . : Yes

C:\Users\hp>
```

Figure 4: ipconfig/All command

- **Ping** ip of other device:

We ping other device, then send 4 packets and the device receive all packets



```
Command Prompt
C:\Users\hp>ping 192.168.0.184

Pinging 192.168.0.184 with 32 bytes of data:
Reply from 192.168.0.184: bytes=32 time=54ms TTL=64
Reply from 192.168.0.184: bytes=32 time=80ms TTL=64
Reply from 192.168.0.184: bytes=32 time=39ms TTL=64
Reply from 192.168.0.184: bytes=32 time=7ms TTL=64

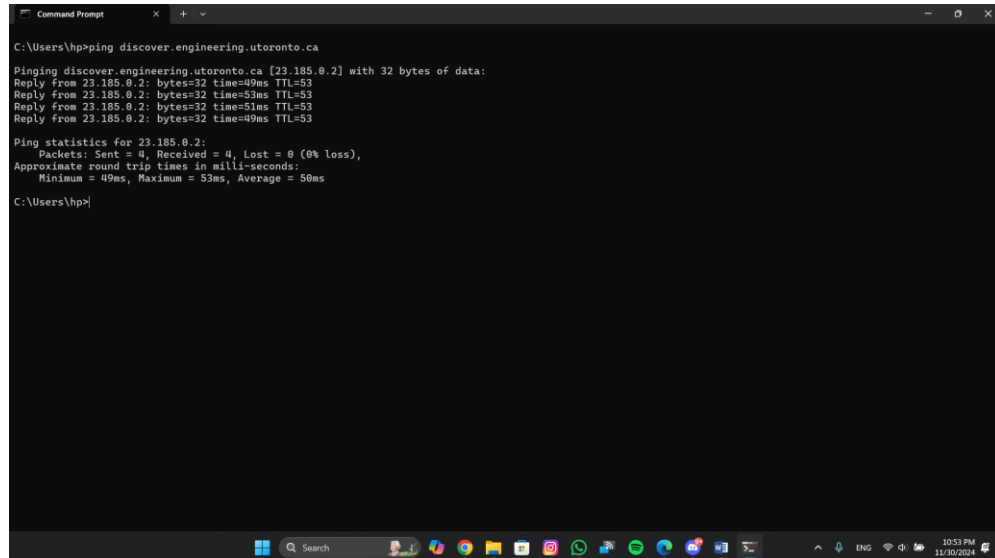
Ping statistics for 192.168.0.184:
    Packets: Sent = 4, Received = 4, Lost = 0 (0% loss),
    Approximate round trip times in milli-seconds:
        Minimum = 7ms, Maximum = 80ms, Average = 45ms

C:\Users\hp>|
```

Figure 5: ping to other device

- **Ping discover.engineering.utoronto.ca**

We tried ping to the website, it sent 4 packets and the website server receive all packets



```
C:\Users\hp>ping discover.engineering.utoronto.ca

Pinging discover.engineering.utoronto.ca [23.185.0.2] with 32 bytes of data:
Reply from 23.185.0.2: bytes=32 time=49ms TTL=53
Reply from 23.185.0.2: bytes=32 time=53ms TTL=53
Reply from 23.185.0.2: bytes=32 time=51ms TTL=53
Reply from 23.185.0.2: bytes=32 time=49ms TTL=53

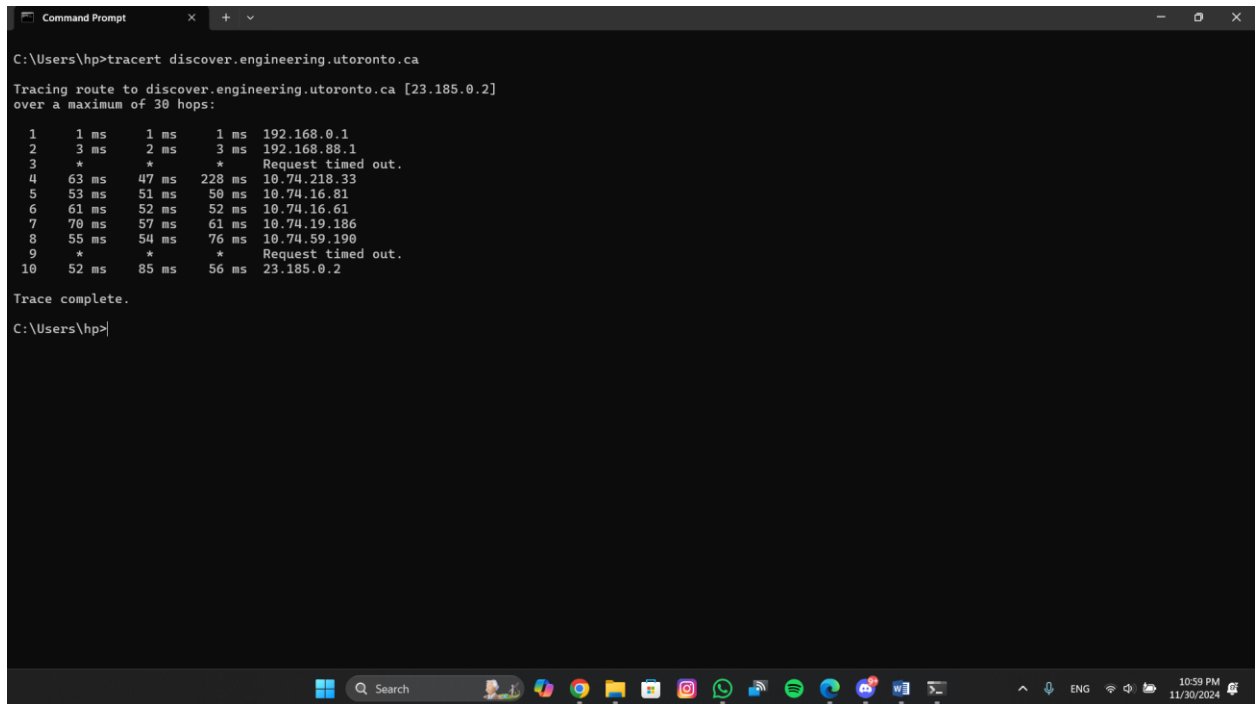
Ping statistics for 23.185.0.2:
    Packets: Sent = 4, Received = 4, Lost = 0 (0% loss),
    Approximate round trip times in milli-seconds:
        Minimum = 49ms, Maximum = 53ms, Average = 50ms

C:\Users\hp>
```

Figure 6: ping to website

- **Tracert discover.engineering.utoronto.ca**

This command will check the path until arrives to server, each line is hop so there are 10 hops to arrives the sever.



```
Command Prompt
C:\Users\hp>tracert discover.engineering.utoronto.ca

Tracing route to discover.engineering.utoronto.ca [23.185.0.2]
over a maximum of 30 hops:
  0  1 ms  1 ms  1 ms  192.168.0.1
  1  3 ms  2 ms  3 ms  192.168.88.1
  2  *      *      *      Request timed out.
  3  63 ms  47 ms  228 ms  10.74.218.33
  4  53 ms  51 ms  50 ms  10.74.16.81
  5  61 ms  52 ms  52 ms  10.74.16.61
  6  70 ms  57 ms  61 ms  10.74.19.186
  7  55 ms  54 ms  76 ms  10.74.59.190
  8  *      *      *      Request timed out.
  9  52 ms  85 ms  56 ms  23.185.0.2

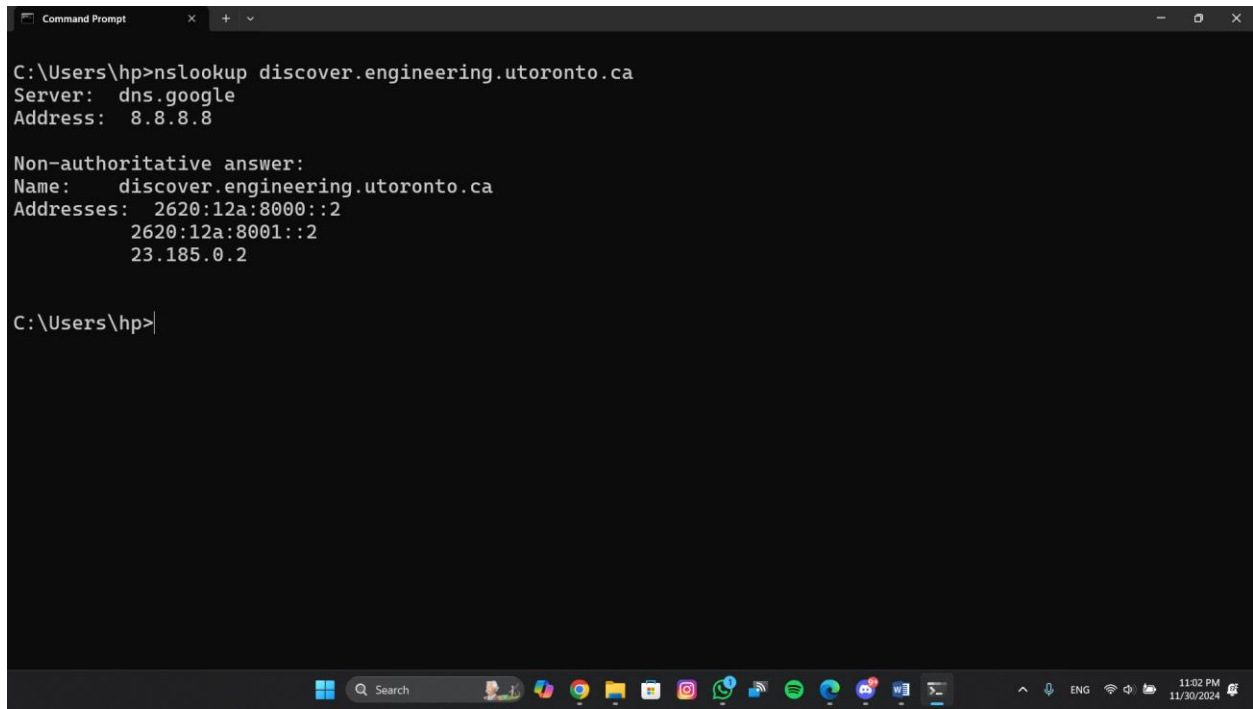
Trace complete.

C:\Users\hp>
```

Figure 7: Tracert command

- **nslookup discover.engineering.utoronto.ca**

this command display some information about this website.



```
Command Prompt
C:\Users\hp>nslookup discover.engineering.utoronto.ca
Server:  dns.google
Address:  8.8.8.8

Non-authoritative answer:
Name:     discover.engineering.utoronto.ca
Addresses: 2620:12a:8000::2
           2620:12a:8001::2
           23.185.0.2

C:\Users\hp>
```

Figure 8: nslookup command

- **telnet discover.engineering.utoronto.ca**

we tried to connect to the website but it give us Bad Request

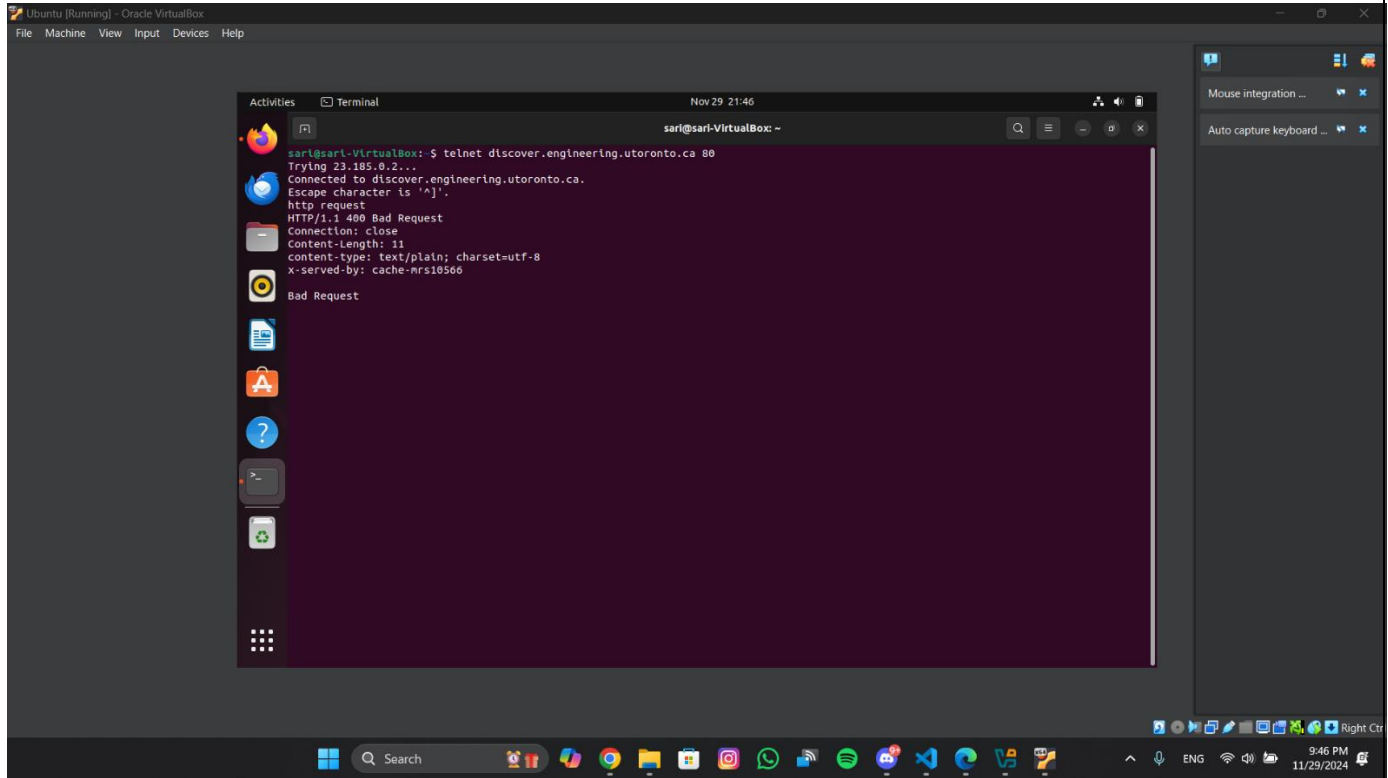


Figure 9: telnet command

### 1.3 Wireshark packet analyzer:

We use the wireshark program to see DNS query to discover.engineering.utoronto.ca and the following figure show the result.

Note that the red line is DNS query:

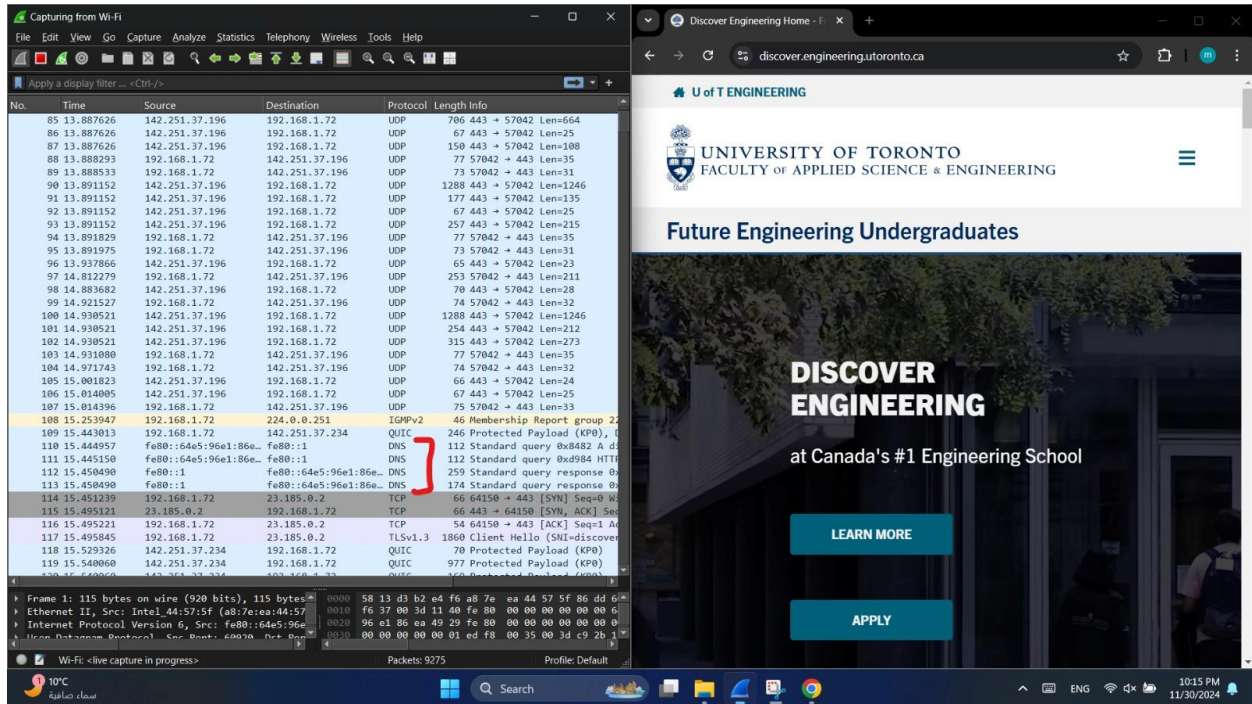


Figure 10: Try wirshark

## 2. Task 2-Web Server:

### 2.1 Socket Setup:

```
1  from socket import *
2  import os
3
4  server_socket = socket(AF_INET, SOCK_STREAM)
5  mainport=5689
6  server_socket.bind(("0.0.0.0", mainport))
7  server_socket.listen(1)
8  |
9  print("server is listening")
10
```

Figure 11: code for socket setup

1. **Initialization of the socket:** The server is initialized using the `socket (AF_INET, SOCK_STREAM)` function where:
  - a) `AF_INET`: Specifies the use of IPv4 addressing.
  - b) `SOCK_STREAM`: Sets up a TCP connection for reliable data transfer.
  - c) This creates a socket object to handle incoming connections.
2. **Binding:** The server binds to a specific IP address and port using the `Server_socket. bind (("0.0.0.0", port))` the "0.0.0.0" makes the server accessible to all network interfaces.
3. **Listening:** `server_socket.listen(1)` enables the server to listen for incoming connections.



## 2.2 Request Handling:

```
11 while True:
12     client_socket, addr = server_socket.accept()
13     ip=addr[0]
14     port = addr[1]
15
16     request = client_socket.recv(1024).decode("utf-8")
17
18     # request = request.decode('utf-8')
19
20     print(request)
21
22     type=request.split("\n")[0]
23     req=type.split(" ")[1]
24
```

Figure 12: code for Request Handling

### 1. Receiving Requests:

- a) To accept incoming client connections all the time using `accept()`, we use while loop, then take the client socket and address (IP and port).

The server receives HTTP requests from clients using `client_socket.recv(1024)` where:

- b) 1024 is the buffer size in bytes for incoming data.
- c) The received request is decoded from bytes to a string using `decode("utf-8")` as required.

### 2. Request Analysis:

- a) The HTTP request string is split into lines using `split("\n")`.
- b) The first line (the request line) contains the HTTP method (like GET) and the requested resource path (like `/main_en.html`).
- c) The requested path is extracted using `type.split(" ")[1]`.

3. **Handling Different Request Types:** The server identifies the type of resource requested Based on the extracted path (HTML files, CSS files, Images) using if-statement.

## 2.3 File Serving:

1. **Dynamic Files Serving:** The server identifies the requested resource path, files are opened in the appropriate mode using if statements:
  - a) Text are opened in read mode with UTF-8 encoding
  - b) Binary files are opened in binary read mode
2. **Sending Files:** The server sends a response header (like **HTTP/1.1 200 OK**) followed by the file content using `client_socket.send()`.
  - a) Html and CSS file loaded at the first of running: -

```
31     if req== "/" or req=="/en" or req=="/index.html" or req=="/main_en.html":
32         client_socket.send("HTTP/1.1 200 OK\r\n".encode())
33         client_socket.send("Content-Type: text/html\r\n".encode())
34         client_socket.send("\r\n".encode())
35         htmlenfile=open("main_en.html","r",encoding="utf-8")
36         client_socket.send(((htmlenfile.read()).encode()))
37     elif req=="/styles.css":
38         client_socket.send("HTTP/1.1 200 OK\r\n".encode())
39         client_socket.send("Content-Type: text/css\r\n".encode())
40         client_socket.send("\r\n".encode())
41         css = open("styles.css", "r", encoding="utf-8")
42         client_socket.send(((css.read()).encode()))
43
```

Figure 13: code for html and css en page loading

b) .Png Images: there are 5 images in our web at the first of running:-

```
45 elif req=="/images/mohmadzaidPfp.png":
46     client_socket.send("HTTP/1.1 200 OK\r\n".encode())
47     client_socket.send("Content-Type: image/png\r\n".encode())
48     client_socket.send("\r\n".encode())
49     mohmadzaidImage=open("images/mohmadzaidPfp.png","rb")
50     client_socket.send((mohmadzaidImage.read()))
51
52 elif req == "/images/mohmadOmarPfp.png":
53     client_socket.send("HTTP/1.1 200 OK\r\n".encode())
54     client_socket.send("Content-Type: image/png\r\n".encode())
55     client_socket.send("\r\n".encode())
56     mohmadomarImage = open("images/mohmadOmarPfp.png", "rb")
57     client_socket.send((mohmadomarImage.read()))
58
59 elif req == "/images/sariPfp.png":
60     client_socket.send("HTTP/1.1 200 OK\r\n".encode())
61     client_socket.send("Content-Type: image/png\r\n".encode())
62     client_socket.send("\r\n".encode())
63     sariImage = open("images/sariPfp.png", "rb")
64     client_socket.send((sariImage.read()))
65
66 elif req == "/images/sum1.jpg":
67     client_socket.send("HTTP/1.1 200 OK\r\n".encode())
68     client_socket.send("Content-Type: image/jpg\r\n".encode())
69     client_socket.send("\r\n".encode())
70     sumImage = open("images/sum1.jpg", "rb")
71     client_socket.send((sumImage.read()))
72
73 elif req == "/images/sum2.jpg":
74     client_socket.send("HTTP/1.1 200 OK\r\n".encode())
75     client_socket.send("Content-Type: image/jpg\r\n".encode())
76     client_socket.send("\r\n".encode())
77     sumImage = open("images/sum2.jpg", "rb")
78     client_socket.send((sumImage.read()))
79
```

Figure 14: code for send all page images

## 2.4 HTML/CSS Part:

In this part we successfully complete the implementation of main English web page (main\_en.html), and then provide an styles.css page that improve the user interface design, in this main page, there are: -

1. Team members section with names, id's, and images which they are implemented as follow: -

```
15 <section class="team">
16 <div class="teamDiv">
17
18 <div class="headText-team">
19 | <h2>TEAM MEMBER</h2>
20 | </div>
21
22 <div class="line">
23
24 <div class="team-member">
25 | 
26 | <h3>Mohammad Omar</h3>
27 | <p>ID: 1221332</p>
28 | <p>Skills: Front-end development</p>
29 | <p>Hobbies: Drawing, Swimming, Watching films</p>
30 | </div>
31
32 <div class="team-member">
33 | 
34 | <h3>Mohammad Zaid</h3>
35 | <p>ID: 1221265</p>
36 | <p>Skills: coding(C,Java,Python)</p>
37 | <p>Hobbies: Gym,Car racing</p>
38 | </div>
39 <div class="team-member">
40 | 
41 | <h3>Sari Abd AL-Ghani</h3>
42 | <p>ID: 1220982</p>
43 | <p>Skills: hacking</p>
44 | <p>Hobbies: Object Balancing</p>
45 | </div>
46 </div>
47 </div>
48 </section>
```

Figure 15: html code for member section

## 2. links divided as follow

- link to supporting material page (supporting\_matireal\_en.html
- Also an links for retag.edu and textbox of our network material which the provided in html using anchor tag as below.
- the last link provided is for testing the error 404 concept, because (anyPage) file is not in our site.

```
111 <section class="externalLinks">
112
113     <a href="https://gaia.cs.umass.edu/kurose_ross/index.php" target="_blank">Textbook Website</a>
114     <a href="https://ritaj.birzeit.edu/" target="_blank">Ritaj Website</a>
115     <a href="supporting material en.html" >supporting page</a>
116     <a href="anyPage" >File Not in Our Site</a>
117
118 </section>
```

Figure 16: html code for the provided links

## 3. A full well-presented topic from the first two chapters of the textbook about transmission links in networks that discuss the difference between Guided and Unguided transmission links with (.png) images for each.

```
55 <section class="topic">
56     <h2>Transmission Links in Networks</h2>
57     <p>Transmission links are an essential component of computer networks. They determine the physical medium used to transfer data
58         between devices. Depending on the type of network and requirements, there are many specifications should considered
59     </p>
60     <ul>
61         <li><b>Bandwidth</b>: The maximum data rate supported by the medium.</li>
62         <li><b>Distance</b>: The length over which data needs to be transmitted without loss.</li>
63         <li><b>Cost</b>: Budget constraints for installation and maintenance.</li>
64         <li><b>Reliability</b>: The stability of the transmission over time.</li>
65     </ul>
66     <p>The transmission links can be classified into two general types:-</p>
67     <div>
68         <h3>Guided Transmission Links</h3>
69         <p>Guided transmission links use physical media to transmit data, such as cables. These links are reliable and commonly used
70             in local area networks (LANs). Below are some common types of guided links:
71         </p>
72         <ol>
73             <li><b>Twisted Pair Cable</b>: Used in telephony and Ethernet networks; inexpensive and widely available.</li>
74             <li><b>Coaxial Cable</b>: Used for cable TV and broadband; offers higher bandwidth compared to twisted pairs.</li>
75             <li><b>Fiber Optic Cable</b>: Uses light signals for data transfer; supports long distances and high speeds.</li>
76         </ol>
77         
78     </div>
79     <div>
80         <h3>Unguided Transmission Links</h3>
81         <p>Unguided transmission links use wireless media for data transmission. These are useful in wide area networks (WANs) and
82             mobile networks. Examples include:
83         </p>
84         <ol>
85             <li><b>Radio Waves</b>: Used for Wi-Fi, cellular networks, and satellite communications.</li>
86             <li><b>Microwaves</b>: Employed in point-to-point links, often for long distances.</li>
87             <li><b>Infrared</b>: Short-range communication, such as remote controls and some wireless devices.</li>
88         </ol>
89     </div>
90 </section>
```

Figure 17: html code for topic

Then, this main page was translated into an Arabic language to provide main\_ar.html. same as supporting\_matireal\_ar.html.

---

Then we implement a supporting material page (supporting\_matireal\_en.html): That contain a simple form of textbox make the user choose the type of the data that he wants to search on it as shown in the figure below.



Figure 18: supporting page

We create this page as follow

```
1  <!DOCTYPE html>
2  <html lang="en">
3  <head>
4      <meta charset="UTF-8">
5      <meta name="viewport" content="width=device-width, initial-scale=1.0">
6
7      <title>Supporting Material</title>
8  </head>
9  <body>
10
11      <form id="requestForm" method="get" >
12          <label for="fileType">Choose a type:</label>
13          <select id="fileType" name="fileType" required>
14              <option value="image">Image</option>
15              <option value="video">Video</option>
16          </select>
17          <br><br>
18          <label for="fileName">File/Topic Name:</label>
19          <input type="text" id="fileName" name="fileName" placeholder="Enter file or topic name" required>
20          <br><br>
21          <button type="submit">Request</button>
22      </form>
23
24
25
26
27 </body>
28 </html>
```

Figure 19: html code for supporting page

This page responsible on the following: -

First, we check if the entered file is in our web or not by using if-elif, in case of videos, our web there are no videos so the output always will be not found and use the case of (found = 0), but the when its come to img case, we have to check if the img in our web or does not found.

When the user search for an object using supporting page, then there are two possible cases:

a) **Found Case:**

in this case we have to check what is the entered data in the textbox by using if statements, for example: -

```
87 elif "/supporting_material_en.html?fileType=image&fileName=" in req:
88
89     nameOfImage=req.split("=")[2]
90     path=os.path.join(os.getcwd(),"images")
91     found=False
92     for filename in os.listdir(path):
93         if filename == nameOfImage:
94             client_socket.send("HTTP/1.1 200 OK\r\n".encode())
95             client_socket.send("Content-Type: image/png\r\n".encode())
96             client_socket.send("\r\n".encode())
97             found=True
98             toOpen="images/"+filename
99             theImage = open(toOpen, "rb")
100             client_socket.send((theImage.read()))
101             print("=====File found=====")
102             break
```

Figure 20: code for find case

This code is one of many if statements in our program (that will explain efficiently in it's part) check if the img entered by the user is in our web or not, in this case we check if the img is mohmadOmarPfp.png img or not, if the condition successfully occurred, then client has requested the image file located at / images/ mohmadOmarPfp.png.

b) **Not Found Case:**

If the requested file is unavailable:

- For images: A "307 Temporary Redirect" sends the client to a Google Image search for the entered term.

```
104 if not found:
105     search_url = f"https://www.google.com/search?q={nameOfImage.replace(' ', '+')}&tbm=isch"
106     response = (
107         "HTTP/1.1 307 Temporary Redirect\r\n"
108         "Location:"+ search_url + "\r\n"
109         "Content-Length: 0\r\n"
110         "Connection: close\r\n"
111         "\r\n"
112     )
113
```

Figure 21: code for not found image case

- ❖ Here if not found (found is a int variable that could be 1 in case of the content available or 0 in case of not found).
- ❖ Then in second line, it's constructed google img search URL that take the entered img name and concatenate it with the URL to provide an ready link.

- ❖ And finally creates an HTTP response that tells the client to redirect to a different URL.
- For videos: A "307 Temporary Redirect" sends the client to a YouTube search.

```
137         if not found:
138             search_url = f"https://www.youtube.com/search?q={nameOFVideo.replace(' ', '+')}&tbm=isch"
139             response = (
140                 "HTTP/1.1 307 Temporary Redirect\r\n"
141                 "Location:" + search_url + "\r\n"
142                 "Content-Length: 0\r\n"
143                 "Connection: close\r\n"
144                 "\r\n"
145             )
146             client_socket.sendall(response.encode('utf-8'))
147
```

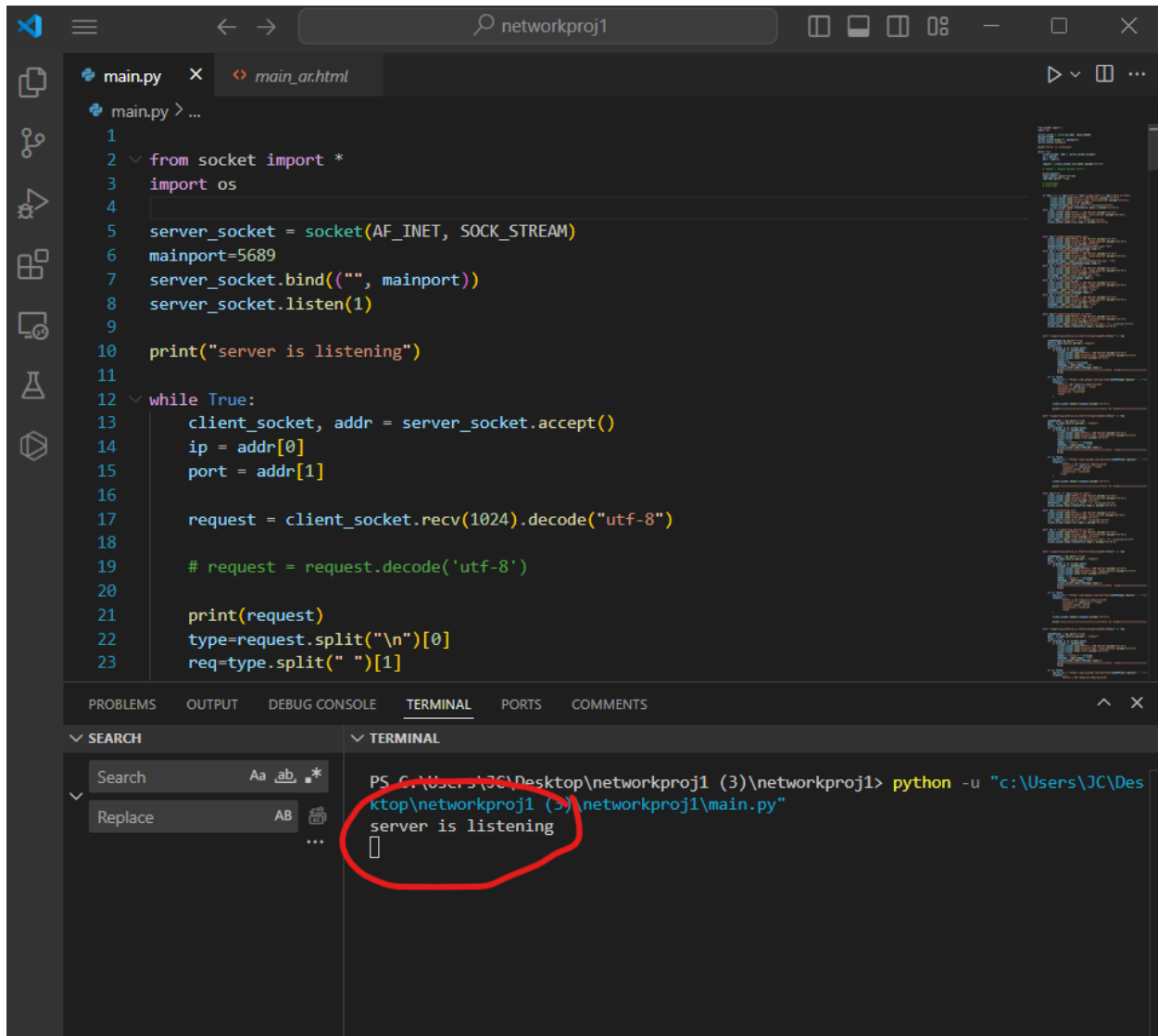
Figure 22: code for not found video case



## 2.5 Run the Cases:

### 2.5.1 load our html web with all obj.

To run the code, first we run the main.py file and then the output will be just like the following: -



The screenshot shows the Visual Studio Code editor with a file named `main.py` open. The code is a Python script that sets up a simple web server using the `socket` module. It binds to port 5689 and listens for incoming connections. A `while` loop continuously accepts connections, receives data, and prints it. The terminal at the bottom shows the command `python -u "c:\Users\JC\Desktop\networkproj1 (3)\networkproj1\main.py"` being executed, and the output `server is listening` is displayed, which is circled in red.

```
1
2 from socket import *
3 import os
4
5 server_socket = socket(AF_INET, SOCK_STREAM)
6 mainport=5689
7 server_socket.bind(("", mainport))
8 server_socket.listen(1)
9
10 print("server is listening")
11
12 while True:
13     client_socket, addr = server_socket.accept()
14     ip = addr[0]
15     port = addr[1]
16
17     request = client_socket.recv(1024).decode("utf-8")
18
19     # request = request.decode('utf-8')
20
21     print(request)
22     type=request.split("\n")[0]
23     req=type.split(" ")[1]
```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS COMMENTS

SEARCH

Replace

PS C:\Users\JC\Desktop\networkproj1 (3)\networkproj1> python -u "c:\Users\JC\Desktop\networkproj1 (3)\networkproj1\main.py"

server is listening

Figure 23: Run the web server

This mean that the server is listening for request using while loop and waiting to get response.

Then when the user enter **localhost:5689** in the browser, the **main\_en.html** file will be loaded, So the user can enter localhost: 5689/main\_en.html or localhost:5689/en will give the following output and this will give (this will run the English version of our web) which handled in the following part of code:-

```

31     if req=="/" or req=="/en" or req=="/index.html" or req=="/main_en.html":
32         client_socket.send("HTTP/1.1 200 OK\r\n".encode())
33         client_socket.send("Content-Type: text/html\r\n".encode())
34         client_socket.send("\r\n".encode())
35         htmlenfile=open("main_en.html","r",encoding="utf-8")
36         client_socket.send(((htmlenfile.read()).encode()))
37     elif req=="/styles.css":
38         client_socket.send("HTTP/1.1 200 OK\r\n".encode())
39         client_socket.send("Content-Type: text/css\r\n".encode())
40         client_socket.send("\r\n".encode())
41         css = open("styles.css", "r", encoding="utf-8")
42         client_socket.send(((css.read()).encode()))
43

```

Figure 24: handle the page request

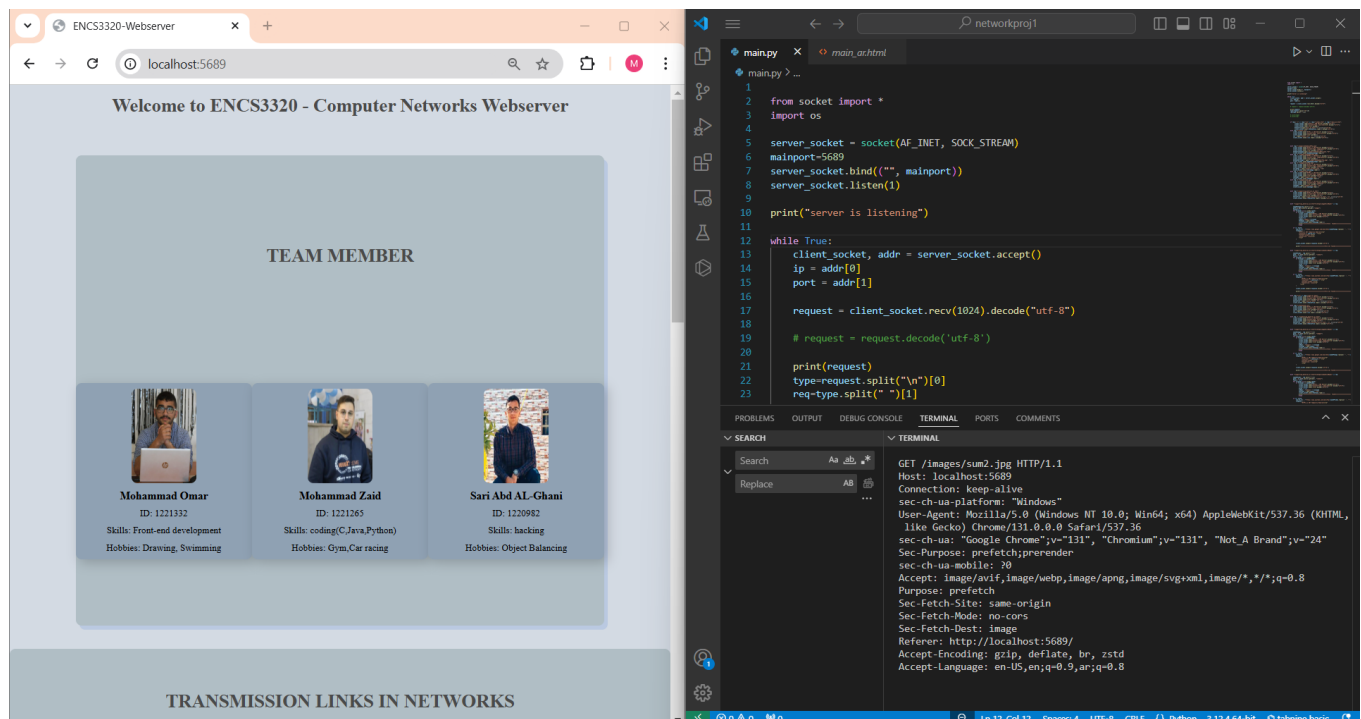


Figure 25: open en website

And if the user enter **localhost:5689/ar** the main\_ar.html file will be loaded, and the output will be as the following: (this will run the Arabic version of our web)

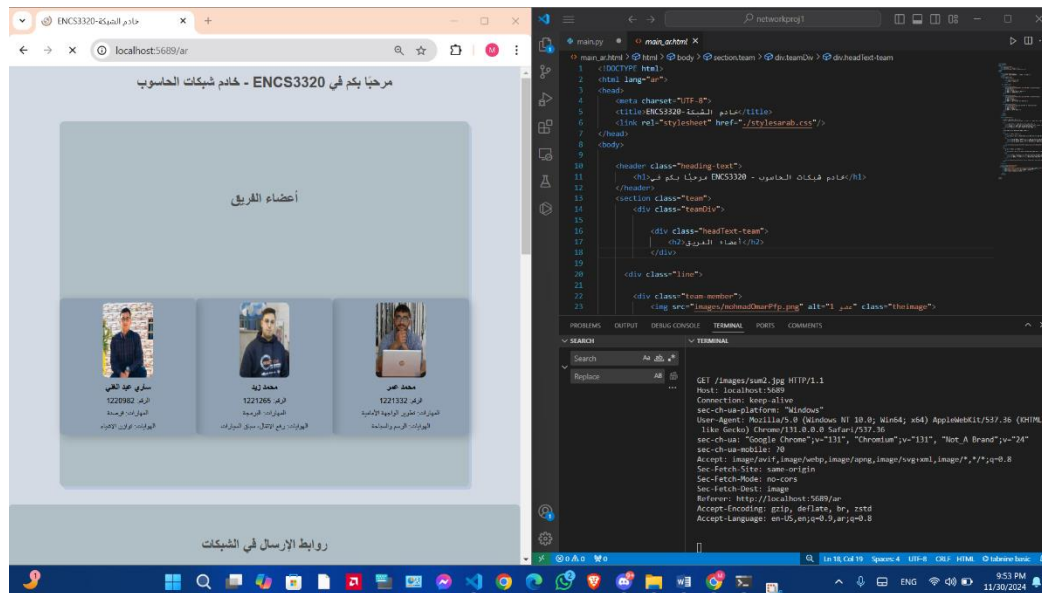


Figure 26: open ar page

In case of the output in the command of the main.py code after run localhost:5689 the output will be:-

```
PS C:\Users\JC\Desktop\networkproj1 (3)\networkproj1> python -u "c:\Users\JC\Desktop\networkproj1 (3)\networkproj1\main.py"
server is listening
GET / HTTP/1.1
Host: localhost:5689
Connection: keep-alive
Cache-Control: max-age=0
sec-ch-ua: "Google Chrome";v="131", "Chromium";v="131", "Not A Brand";v="24"
sec-ch-ua-mobile: ?0
sec-ch-ua-platform: "Windows"
Upgrade-Insecure-Requests: 1
User-Agent: Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/131.0.0.0 Safari/537.36
Accept: text/html,application/xhtml+xml,application/xml;q=0.9,image/avif,image/webp,image/apng,*/*;q=0.8,application/signed-exchange;v=b3;q=0.7
Sec-Fetch-Site: none
Sec-Fetch-Mode: navigate
Sec-Fetch-User: ?1
Sec-Fetch-Dest: document
Accept-Encoding: gzip, deflate, br, zstd
Accept-Language: en-US,en;q=0.9,ar;q=0.8

GET /styles.css HTTP/1.1
Host: localhost:5689
Connection: keep-alive
sec-ch-ua-platform: "Windows"
User-Agent: Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/131.0.0.0 Safari/537.36
sec-ch-ua: "Google Chrome";v="131", "Chromium";v="131", "Not A Brand";v="24"
sec-ch-ua-mobile: ?0
Accept: text/css,*/*;q=0.1
Sec-Fetch-Site: same-origin
Sec-Fetch-Mode: no-cors
Sec-Fetch-Dest: style
Referer: http://localhost:5689/
Accept-Encoding: gzip, deflate, br, zstd
Accept-Language: en-US,en;q=0.9,ar;q=0.8

GET /images/mohammedarPfp.png HTTP/1.1
Host: localhost:5689
Connection: keep-alive
sec-ch-ua-platform: "Windows"
User-Agent: Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/131.0.0.0 Safari/537.36
sec-ch-ua: "Google Chrome";v="131", "Chromium";v="131", "Not A Brand";v="24"
sec-ch-ua-mobile: ?0
Accept: image/avif,image/webp,image/apng,image/svg+xml,image/*,*/*;q=0.8
Sec-Fetch-Site: same-origin
Sec-Fetch-Mode: no-cors
Sec-Fetch-Dest: image
Referer: http://localhost:5689/
Accept-Encoding: gzip, deflate, br, zstd
Accept-Language: en-US,en;q=0.9,ar;q=0.8
```

Figure 27: http request output on the terminal 1

```

GET /images/mohmadzaidPfp.png HTTP/1.1
Host: localhost:5689
Connection: keep-alive
sec-ch-ua-platform: "Windows"
User-Agent: Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/131.0.0.0 Safari/537.36
sec-ch-ua: "Google Chrome";v="131", "Chromium";v="131", "Not_A_Brand";v="24"
sec-ch-ua-mobile: ?0
Accept: image/avif,image/webp,image/apng,image/svg+xml,image/*,*/*;q=0.8
Sec-Fetch-Site: same-origin
Sec-Fetch-Mode: no-cors
Sec-Fetch-Dest: image
Referer: http://localhost:5689/
Accept-Encoding: gzip, deflate, br, zstd
Accept-Language: en-US,en;q=0.9,ar;q=0.8

GET /images/saripfp.png HTTP/1.1
Host: localhost:5689
Connection: keep-alive
sec-ch-ua-platform: "Windows"
User-Agent: Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/131.0.0.0 Safari/537.36
sec-ch-ua: "Google Chrome";v="131", "Chromium";v="131", "Not_A_Brand";v="24"
sec-ch-ua-mobile: ?0
Accept: image/avif,image/webp,image/apng,image/svg+xml,image/*,*/*;q=0.8
Sec-Fetch-Site: same-origin
Sec-Fetch-Mode: no-cors
Sec-Fetch-Dest: image
Referer: http://localhost:5689/
Accept-Encoding: gzip, deflate, br, zstd
Accept-Language: en-US,en;q=0.9,ar;q=0.8

GET /images/sum1.jpg HTTP/1.1
Host: localhost:5689
Connection: keep-alive
sec-ch-ua-platform: "Windows"
User-Agent: Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/131.0.0.0 Safari/537.36
sec-ch-ua: "Google Chrome";v="131", "Chromium";v="131", "Not_A_Brand";v="24"
sec-ch-ua-mobile: ?0
Accept: image/avif,image/webp,image/apng,image/svg+xml,image/*,*/*;q=0.8
sec-ch-ua-platform: "Windows"
User-Agent: Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/131.0.0.0 Safari/537.36
sec-ch-ua: "Google Chrome";v="131", "Chromium";v="131", "Not_A_Brand";v="24"
sec-ch-ua-mobile: ?0
Accept: image/avif,image/webp,image/apng,image/svg+xml,image/*,*/*;q=0.8
Sec-Fetch-Site: same-origin
Sec-Fetch-Mode: no-cors
Sec-Fetch-Dest: image
Referer: http://localhost:5689/
Accept-Encoding: gzip, deflate, br, zstd
Accept-Language: en-US,en;q=0.9,ar;q=0.8

```

Figure 28: http request output on the terminal 2

Every single part of the output is for a single obj in our web (image, .CSS/.html file .....). If we zoom out the case of the CSS file:-

```

GET /styles.css HTTP/1.1
Host: localhost:5689
Connection: keep-alive
sec-ch-ua-platform: "Windows"
User-Agent: Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/131.0.0.0 Safari/537.36
sec-ch-ua: "Google Chrome";v="131", "Chromium";v="131", "Not_A_Brand";v="24"
sec-ch-ua-mobile: ?0
Accept: text/css,*/*;q=0.1
Sec-Fetch-Site: same-origin
Sec-Fetch-Mode: no-cors
Sec-Fetch-Dest: style
Referer: http://localhost:5689/
Accept-Encoding: gzip, deflate, br, zstd
Accept-Language: en-US,en;q=0.9,ar;q=0.8

```

Figure 29: css request

In our code we split the lines to take only the first line, and then we take the second element in the line using split according to (\) to take the file name and then load it, and this criteria is for the whole obj in the web.

We try to run our code in different device (mobile) and it's work successfully by enter the IP of the device that contain the main.py and the port number in the mobile browser.

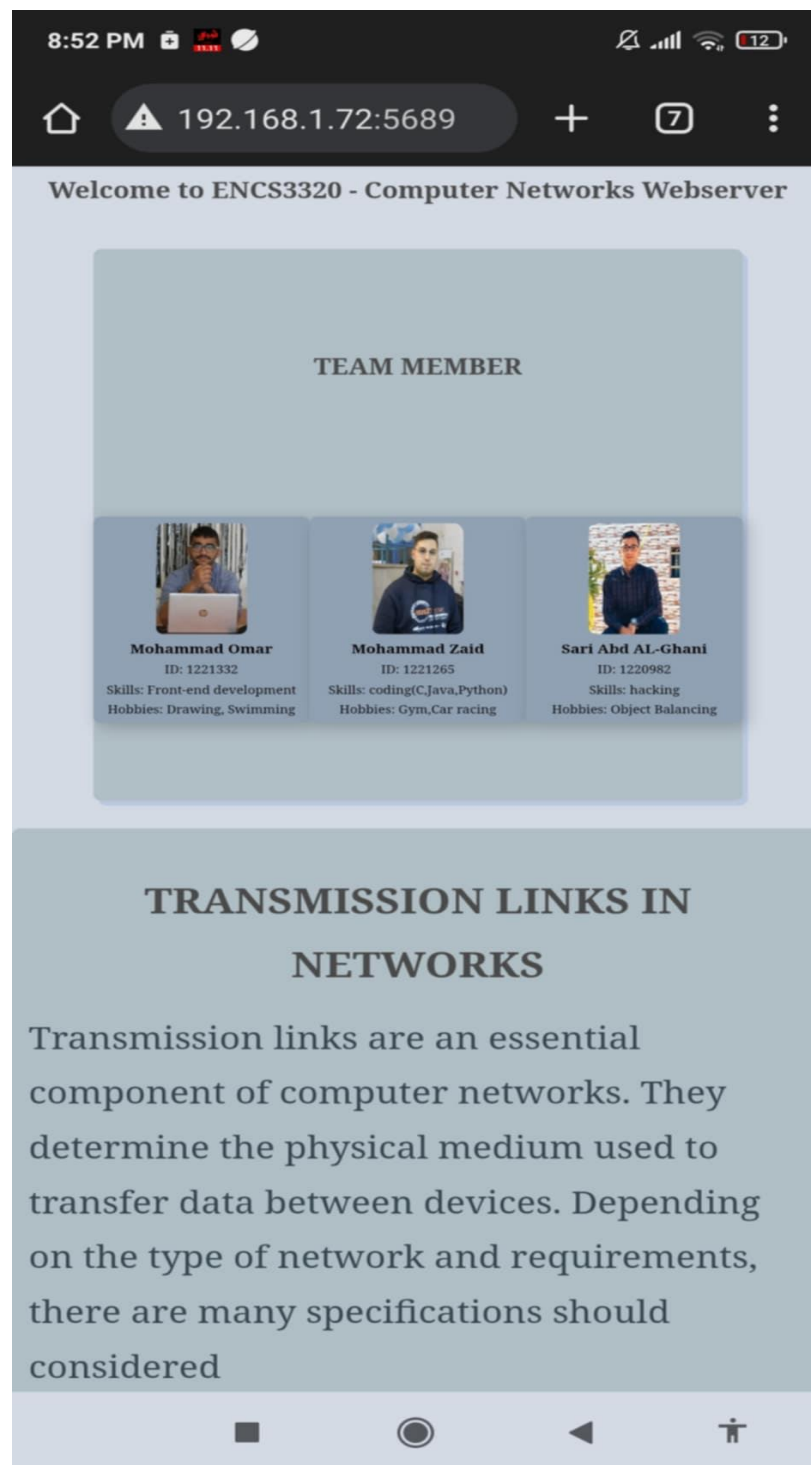


Figure 30: open the website on other devise

## Supporting page Sample Run: -

After running our web as the previous sample run, we go to the supporting page and search for an obj in our web, by choosing the type of the obj (image, video) and then search for the obj name by click Request, in this case the obj name is **mohmadZaidPfp.png** which is an image as shown in the following picture:-

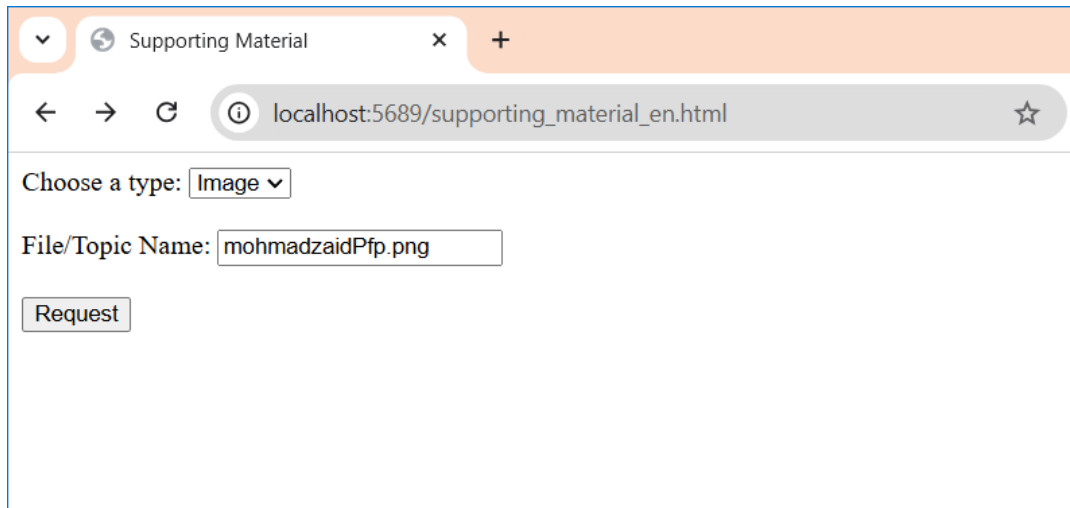


Figure 31: test for founded image

Then our main.py will check if the obj is found or not, in this case the obj is found so the output was successfully shown as in the following image

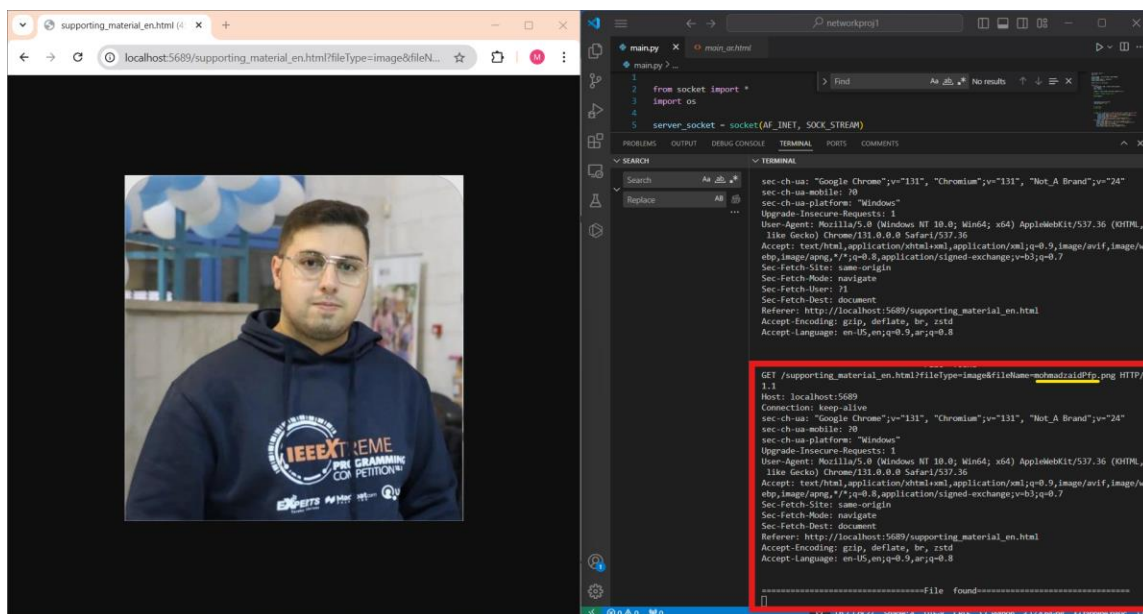


Figure 32: output for founded image

In case of searching for unfounded element, here is a sample run: -

The screenshot shows a web browser window with a single tab titled 'Supporting Material'. The address bar displays 'localhost:5689/supporting\_material\_en.html'. Below the address bar, there is a 'Choose a type:' dropdown menu set to 'Image'. Underneath, a 'File/Topic Name:' text input field contains the value 'test.png'. At the bottom left, there is a button labeled 'Request'.

Figure 33: test for not founded image

Here we entered test.png which is not in our web, so as the required, the output should be as following image

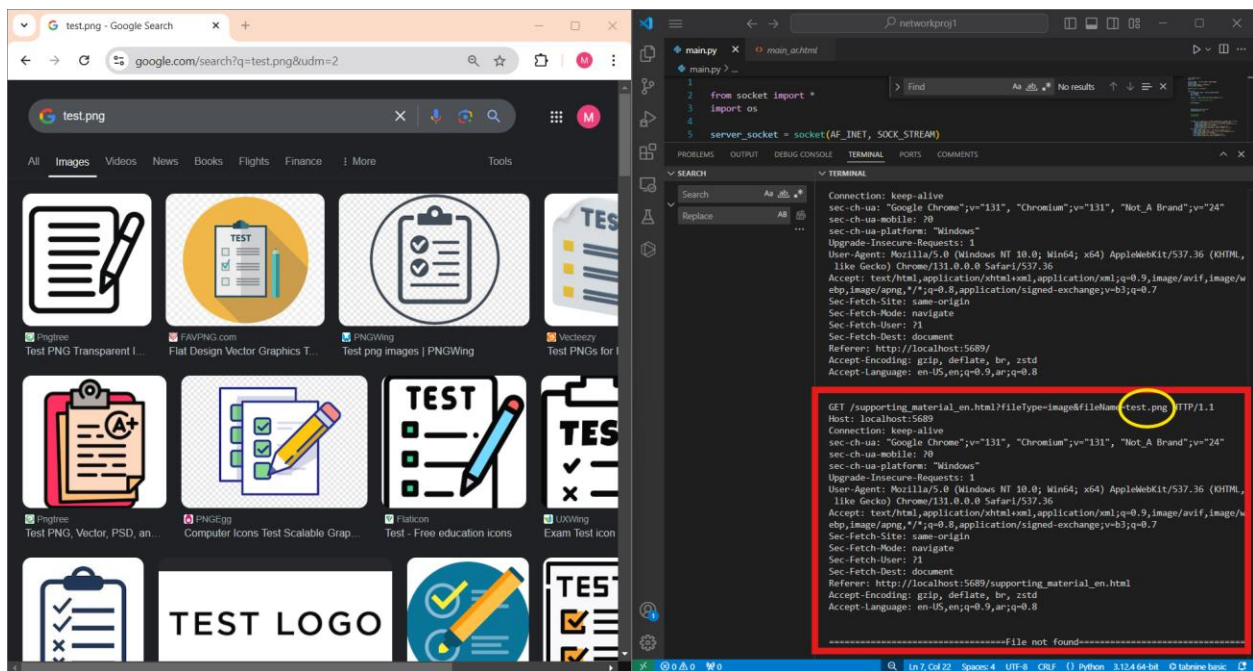


Figure 34: output for not founded image



In case of searching for video, here is the output in case of the video is founded, the “debug.mp4” video is including in video directory in our project file.

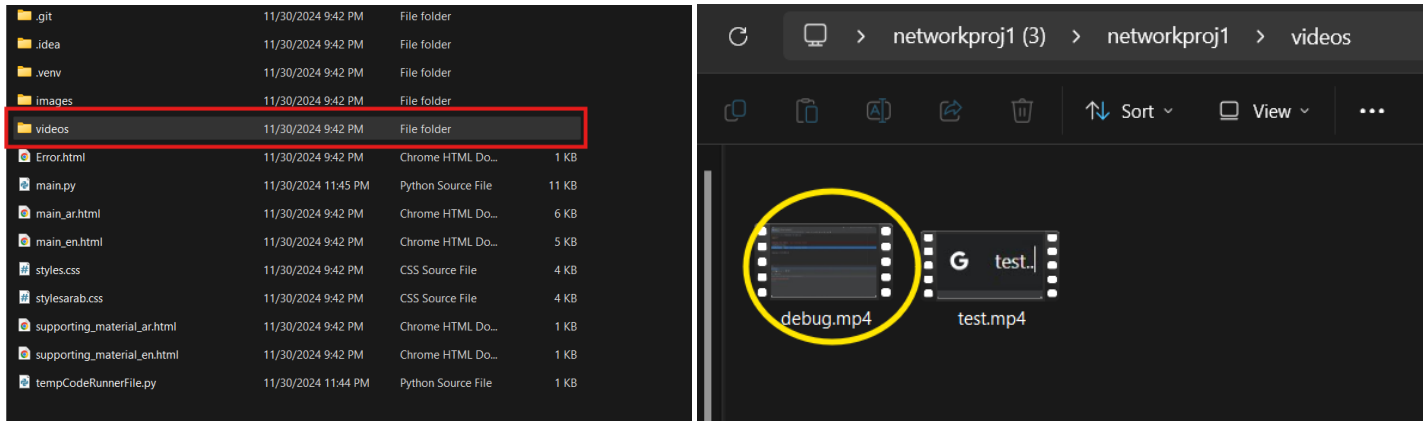


Figure 35: video directory in our project file

And the output will be as the following

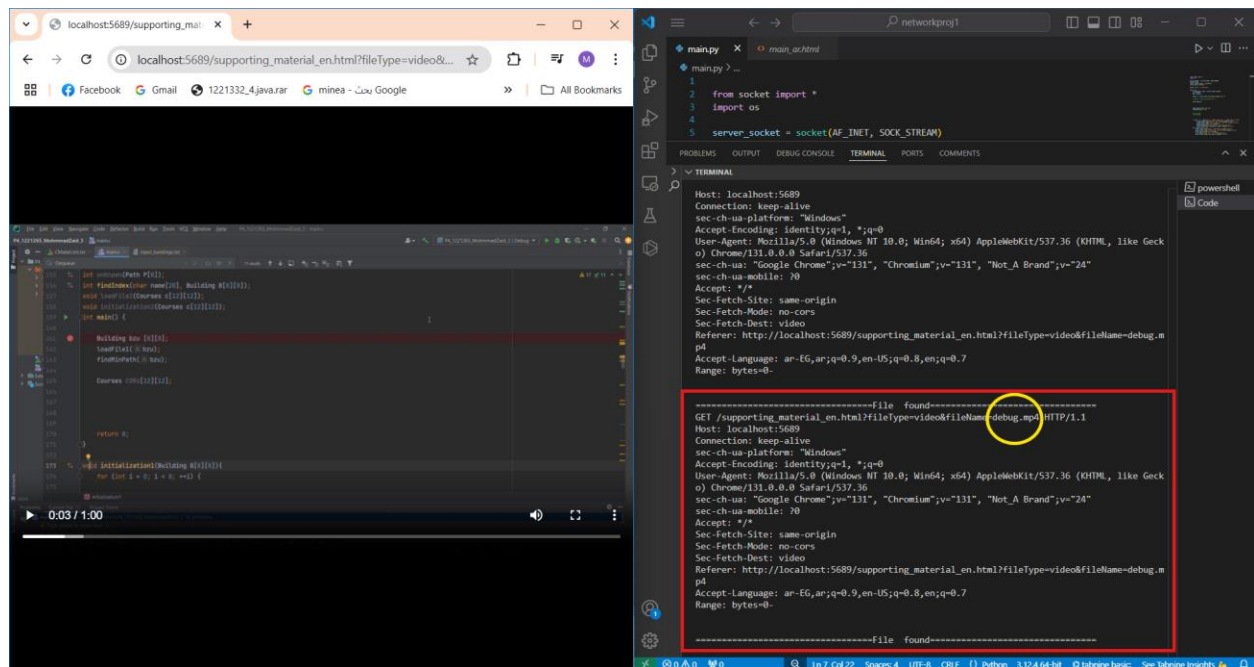


Figure 36: output for founded video



In case of searching for unfounded video, we do the same as the previous run case, but enter Palestine as the name of the video which is not in our project file so the web should take it to YouTube and then show the output as the following

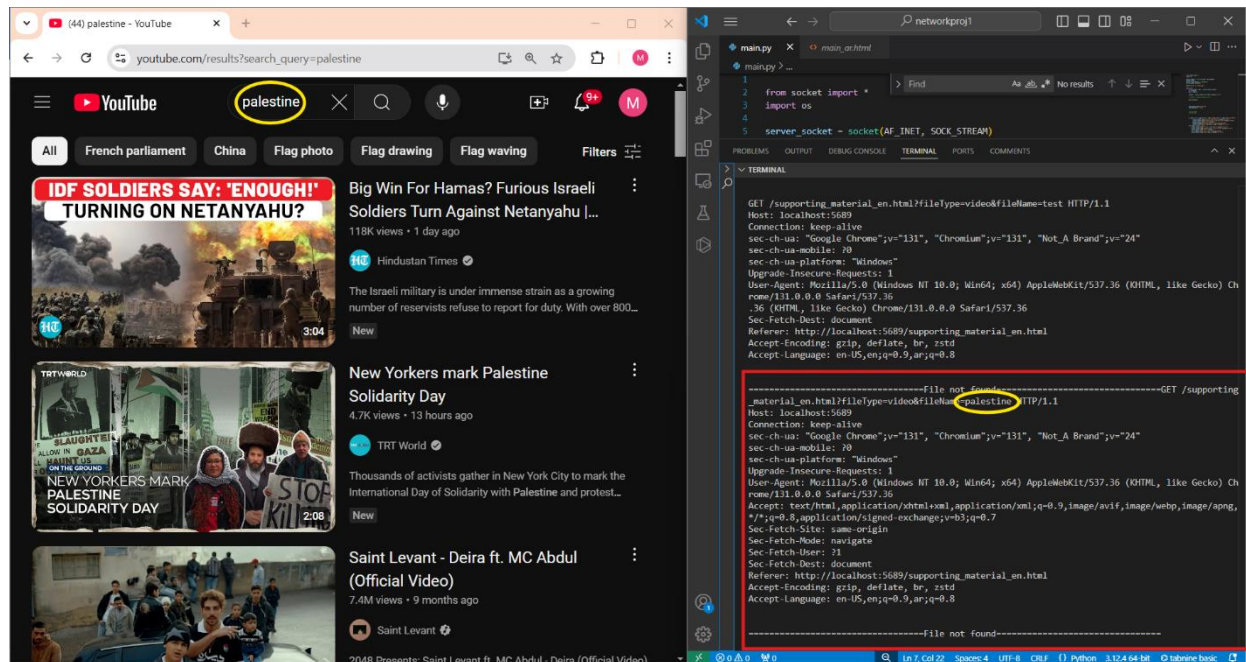


Figure 37:output for not founded video

### 3. Task 3 – UDP Client-Server Trivia Game Using Socket Programming

#### 3.1 What is the Task?

In this Task we will implement an interactive multiplayer trivia game using UDP socket programming to test players' (i.e., clients') knowledge in a fun and competitive environment, the game consists many of rounds, the server will choose set of random question from file and send to the clients. Each client is a player.

##### **The rules of the game:**

- 1- the game starts the number of clients more than or equal 2.
  - 2- Each player has 30 second to answer the question.
  - 3- In each round the server will display 3 question
  - 4- First player answer will get 1 point.
  - 5- The players who write the right answer will get different points according to the time.
  - 6- If the player doesn't answer all question in a round, then the server will eliminate him.
- 

#### 3.2 Run the UDP server

We will try the game and test the code, in addition to explanation each photo:

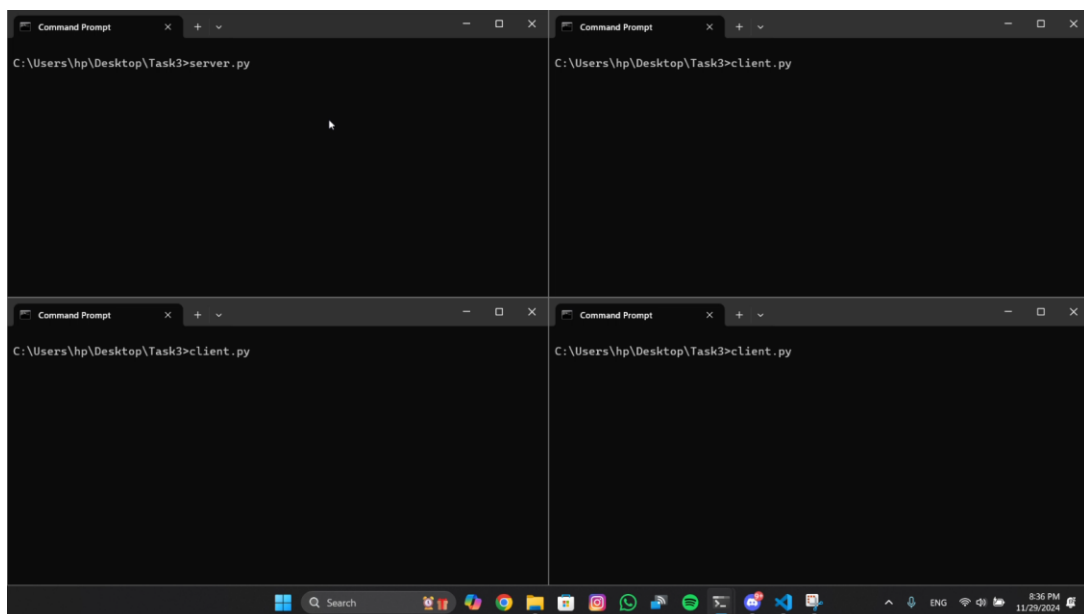


Figure 38: write the command the run the server and the clients

We were open the cmd and write this command to run the server and to join some players.

```
1  import socket
2  import threading
3  import time
4  import random
5
6  # UDP SERVER
7  server = socket.socket(socket.AF_INET, socket.SOCK_DGRAM)
8  PORT = 5689
9  IP = socket.gethostname(socket.gethostname())
10 print(socket.gethostname())
11 print(IP)
12 server.bind((IP, PORT))
13 FORMAT = 'utf-8'
14
```

Figure 39: code of initialize run of UDP server

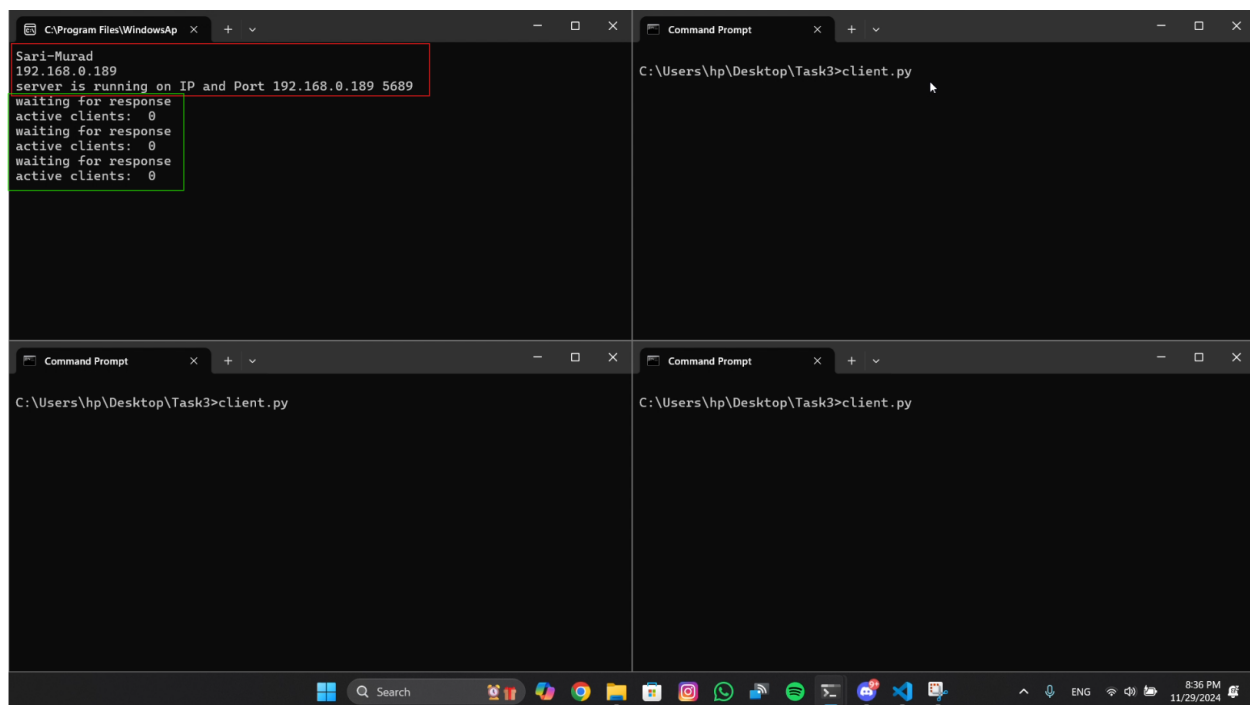


Figure 40: first message from server when run

As shown in the Red box the server prints the name of the device and his IP address and port on the screen, in the line 7 in the code we initialize that the server UDP connection, in line 8 we define a port ( 5689 ) that the players will connect to it, in lines 9-11 we get the name of the device and print the name, IP and port of device who run server code.

```

174 def server_run():
175     print("server is running on IP and Port",IP,PORT)
176     questionInFile = readQuestion()
177     generateNumberOfQuestion(questionInFile)
178     global numberOfQuestion
179     global CounterForQuestion
180     global minNumberToStart
181     global timeBetweenRounds
182     global timeBetweenQuestions
183     global roundQuestions
184     global theRightAnswer
185     global activeClients
186     global nameOfClient
187     try:
188         while True:
189             try:
190                 server.settimeout(2)
191                 message, address = server.recvfrom(1024)
192
193

```

Figure 41: code for start run UDP server

```

282         except socket.timeout:
283             print("waiting for response")
284             print("active clients: ", len(activeClients))

```

Figure 42: code for time out without response

This code for the green box from Figure 38 in the line 188 the sever join infinite loop and try if there any client sends to join, after time out we handle the error in line 282-284 the sever will print number of active clients.

In Figure 39 line 176 we call a function that read a set of question and their answers, these questions we put them in the .txt file and set them into array and calculate the number of question as shown in bellow code:

```

31 def readQuestion(questionInFile=0):
32     file = open('question.txt', 'r')
33     while True:
34         Q = file.readline().strip()
35         A = file.readline().strip()
36         if not Q and not A:
37             break
38
39         questionInFile += 1
40         question.append((Q, A))
41     file.close()
42     return questionInFile
43
44

```

Figure 43: code for server to read the questions

In Figure 39 line 177 we choose 3 questions randomly from all questions, the server will send this questions to the clients in the first round, in addition the server will call this function for the other rounds but in different line, we write the bellow code for choose questions:

```
45 def generateNumberOfQuestion(questionInFile):
46     for i in range(numberOfQuestion):
47         random_number = random.randint(0, questionInFile-1)
48         roundQuestions.append([str(question[random_number][0]), str(question[random_number][1])])
49
50
```

Figure 44: code for server to choose random set of question

### 3.3 Join Player's ( Client's ):

First we write this command in cmd ( client.py ) to run the code were client.py is the name of file of python code.

The bellow Figure show that when any player want to join we ask them to enter their player name and the IP of the server device and the port:

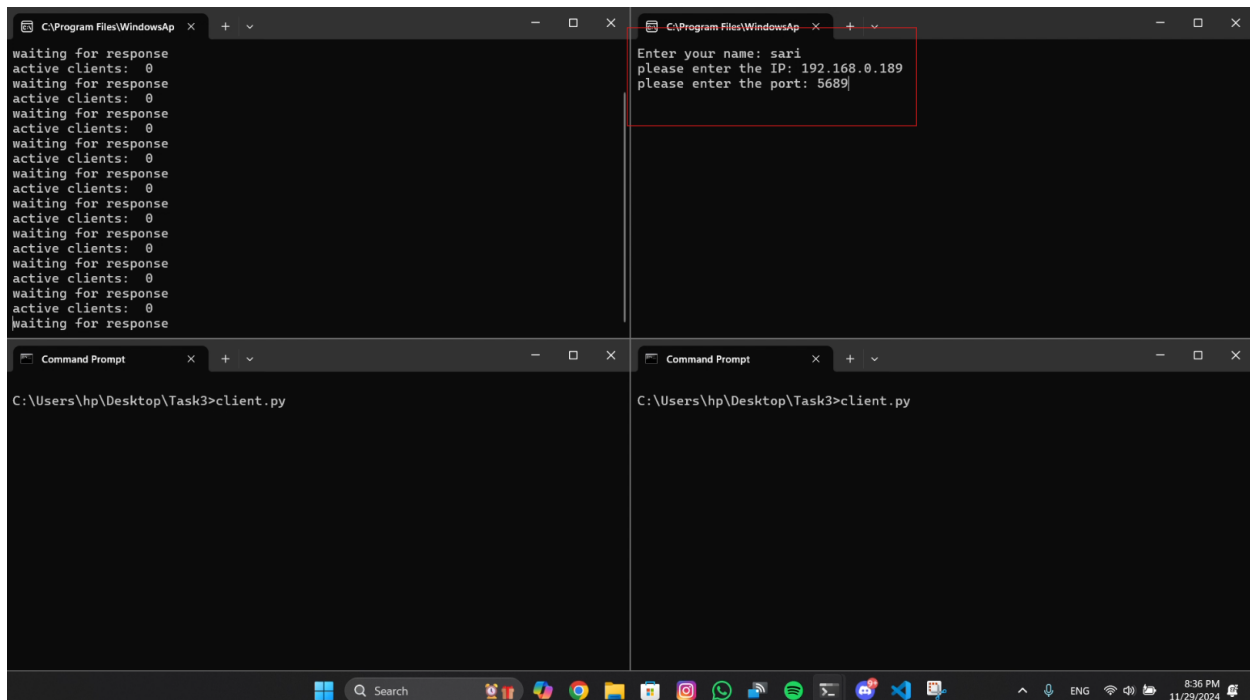


Figure 45: when we want to join player

After we press ENTER key the client will send his IP and Port to the server and the server should announce that a player joins and show his IP, Port and player name, in the bellow figure the server allow to the player to join and the server still waiting another player to start the game:

Note: the IP of player same as server because they are in the same device.

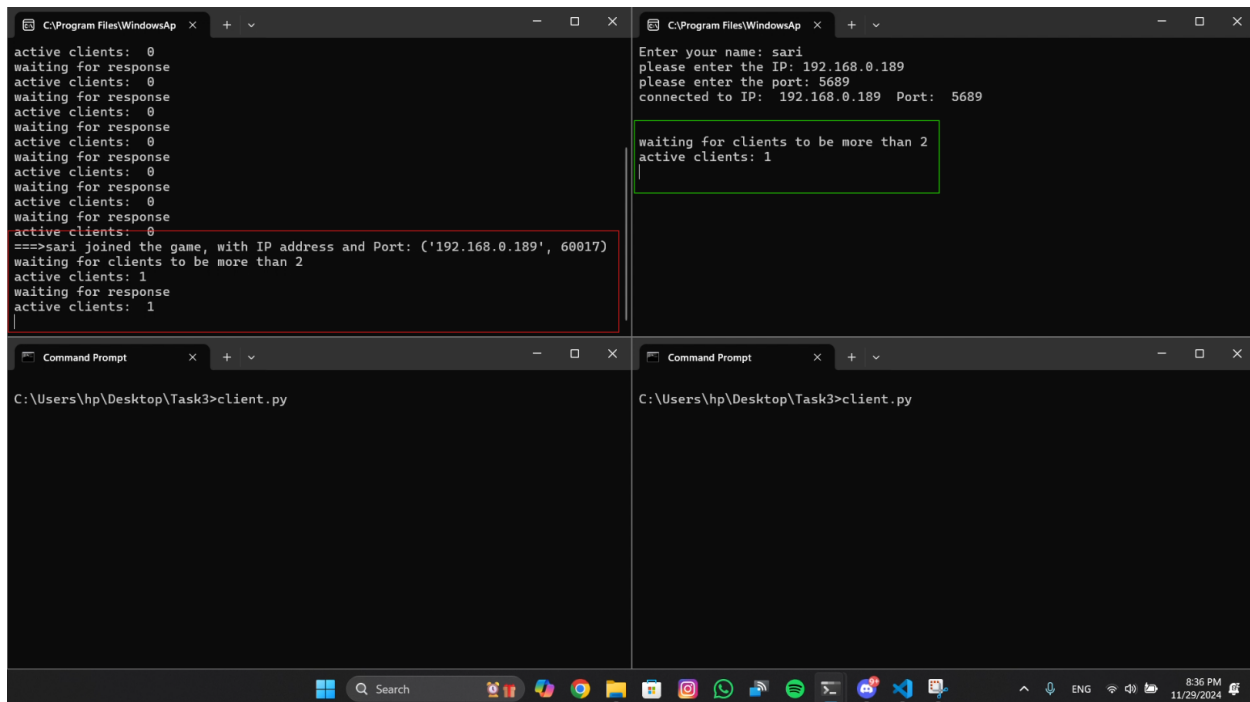


Figure 46: The sever announce the player join and wait

The following code for the client code show how the we ask the user and how we send the message to the server, in addition, how we received a message from user contain the number of active client:

```

1  import socket
2  import threading
3  import time
4
5  FORMAT = 'utf-8'
6
7  client = socket.socket(socket.AF_INET, socket.SOCK_DGRAM)
8  name = input("Enter your name: ")
9
10 IPserver = input("please enter the IP: ")
11 IPserver=str(IPserver)
12
13 PORT = int(input("please enter the port: "))
14
15 joiningMessage = "J@" + name
16 client.sendto(joiningMessage.encode(FORMAT), (IPserver, PORT))
17 print("connected to IP: ", IPserver," Port: ", PORT)
18 lock = threading.Lock()
19

```

Figure 47: code for client to ask the player about IP,port and his name

```

91 def client_run():
92     try:
93         while True:
94             serverMessage = client.recvfrom(1024)[0].decode(FORMAT)
95             type = serverMessage.split("@")[0]
96             thread = threading.Thread(target=handle_connection, args=(serverMessage,))
97             thread.start()
98             if type == "E":
99                 print("\n")
100                 print("GAME OVER!")
101                 break
102         except KeyboardInterrupt:
103             print("\nKeyboardInterrupt detected. Exiting gracefully...")
104         except OSError:
105             print("OFF")
106         except Exception as e:
107             print(f"\nAn error occurred: {e}")
108         finally:
109             pass
110
111 if __name__ == "__main__":
112     client_run()

```

Figure 48: receive message from sever

The client should listen to the server in any time the server sends anything so we make a code to be multithreads, the main process should listen to the sever for infant as shown in the Figure -- but the thread will handle any message from server.

The bellow code from server show how the server handle the player message and sent to the announce the Player information in the server screen, in addition to send message to the player contain number of active players:

```

109 v def handle_client(message, address):
110     clientMessage=message.decode(FORMAT)
111     type=clientMessage.split("@")[0]
112
113
114 v if type=="J":
115     data = clientMessage.split("@")[1]
116 v     if address not in activeClients:
117 v         for client in nameOfClient:
118             joinMessage = "J@"+data+" with address: "+str(address)+"@" + str(len(activeClients))
119             server.sendto(joinMessage.encode(FORMAT), client[1])
120             nameOfClient.append([data, address, 0, 0.0, 0.0,True])
121             activeClients.append(address)
122             print("==>" + data + " joined the game" + ", with IP address and Port: " + str(address))
123

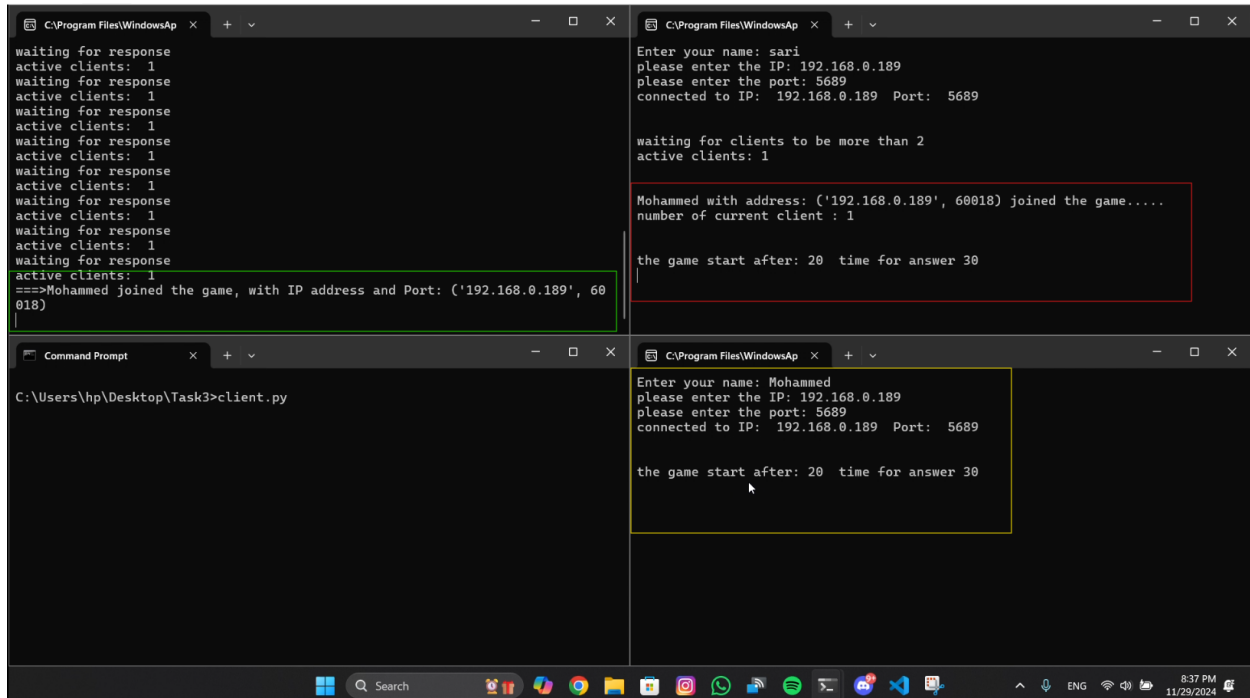
```

Figure 49:code for server handle request to join and send message

**Note:** when we send the message from client to server if the message for join, then we insert “J@” at the first of the message to make it easier for the server distinguish between messages.

### 3.4 Start the Game:

same as the above steps we make another player join, When the number of players become 2 then The server sends a message that the game will start after 20s as shown bellow:



```
waiting for response
active clients: 1
waiting for response
active clients: 1
waiting for response
active clients: 1
waiting for response
active clients: 1
waiting for response
active clients: 1
waiting for response
active clients: 1
waiting for response
active clients: 1
===>Mohammed joined the game, with IP address and Port: ('192.168.0.189', 60018)

Enter your name: sari
please enter the IP: 192.168.0.189
please enter the port: 5689
connected to IP: 192.168.0.189 Port: 5689

waiting for clients to be more than 2
active clients: 1

Mohammed with address: ('192.168.0.189', 60018) joined the game....
number of current client : 1

the game start after: 20 time for answer 30

C:\Users\hp\Desktop\Task3>client.py

Enter your name: Mohammed
please enter the IP: 192.168.0.189
please enter the port: 5689
connected to IP: 192.168.0.189 Port: 5689

the game start after: 20 time for answer 30
```

Figure 50: 2 players join

The following code from server code show that the server sends a message for all players, the message contains the time to send the question in addition to the time for answering:

```
198 while len(activeClients) >= minNumberToStart:
199
200     counterForQuestion = 0
201     for client in activeClients:
202         startingMeessage = "s@" + "the game start after: " + "@" + str(timeBetweenQuestions) + " time for answer " + "@" + str(timeForResponse)
203         server.sendto(startingMeessage.encode(FORMAT), client)
204
205     while counterForQuestion < numberOfQuestion :
```

Figure 51: server code when the number of player become 2



**Note:** we insert at the first of message “S@” to make it easier to handle the message in client code.

The client handles the message to start be create thread and split the text according to @, if the first of message S then the message is start message, then we print the message:

```
33     elif type == "S":
34         text = serverMessage.split("@")[1]
35         timeToStart = serverMessage.split("@")[2]
36         timeForAnswer = serverMessage.split("@")[3]
37         print("\n")
38         print(text + timeToStart + timeForAnswer)
39
```

Figure 52: client receive the start message

After that the server will start send the questions for the players, and receive answers from them as shown in the below Figure.

At the top-right side the player name is: **sari**

At the bottom-right side the player name is: **Mohammed**

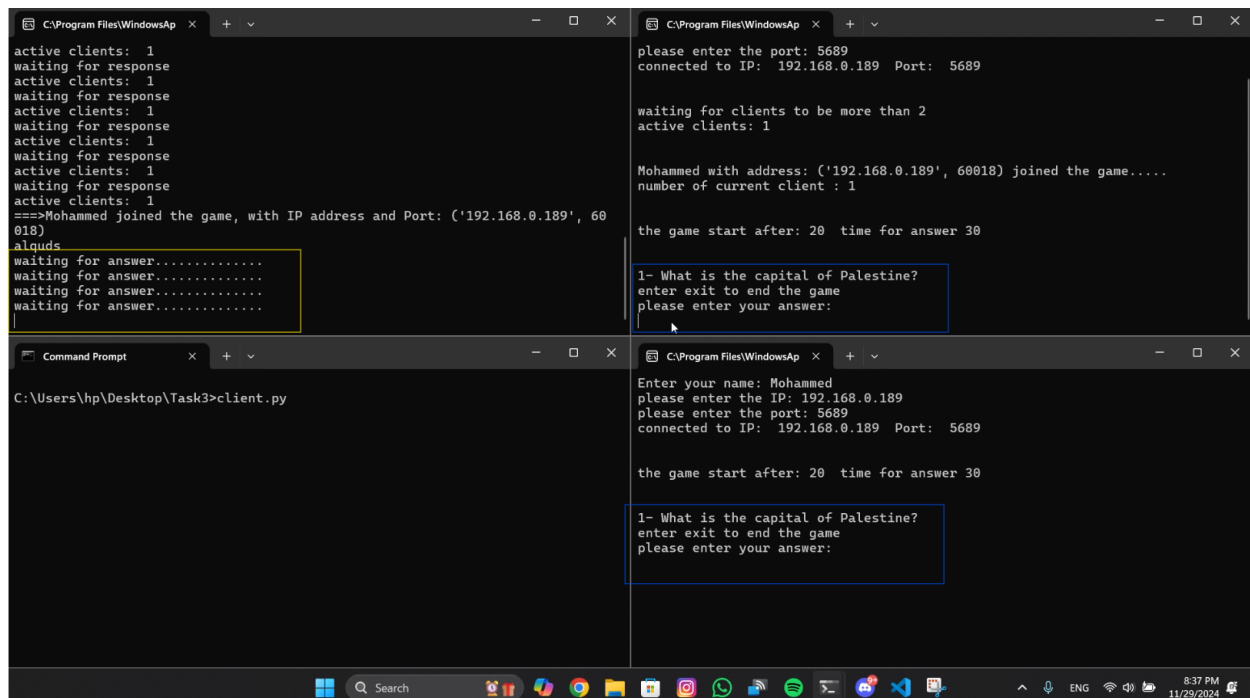


Figure 53: Server send the question and wait the answer

**Note:** the Server insert “Q@” at the first of message to make it easier for the client to handle the question.

```
206 while counterForQuestion < numberOfQuestion :
207
208     time.sleep(timeBetweenQuestions)
209     theQuestion = roundQuestions[counterForQuestion][0]
210     theRightAnswer = roundQuestions[counterForQuestion][1]
211     print(theRightAnswer)
212     sendNumber=counterForQuestion+1
213     questionMessage = "Q@"+str(sendNumber)+"- " + theQuestion
214     for client in activeClients:
215         server.sendto(questionMessage.encode(FORMAT), client)
216
```

Figure 54: code for Server to send the question to all players

Now the client will receive the Question from the server and answering it. When the client answers the question the client will send answer to the server to check if it true or not as shown in the Figure below:

```
C:\Program Files\WindowsAp x + v
waiting for answer.....
waiting for answer.....
alquds who answer it sari ('192.168.0.189', 60017) Right answer
waiting for answer.....
waiting for answer.....
waiting for answer.....
waiting for answer.....
waiting for answer.....
waiting for answer.....
waiting for answer.....
waiting for answer.....
waiting for answer.....
waiting for answer.....
alquds who answer it Mohammed ('192.168.0.189', 60018) Right answer
waiting for answer.....
waiting for answer.....

C:\Program Files\WindowsAp x + v
waiting for clients to be more than 2
active clients: 1

Mohammed with address: ('192.168.0.189', 60018) joined the game....
number of current client : 1

the game start after: 20 time for answer 30

1- What is the capital of Palestine?
enter exit to end the game
please enter your answer:
alquds
enter exit to end the game
please enter your answer:

C:\Program Files\WindowsAp x + v
Enter your name: Mohammed
please enter the IP: 192.168.0.189
please enter the port: 5689
connected to IP: 192.168.0.189 Port: 5689

the game start after: 20 time for answer 30

1- What is the capital of Palestine?
enter exit to end the game
please enter your answer:
Alquds
enter exit to end the game
please enter your answer:
```

Figure 55: the players answer first question

Note that sari and Mohammed both answered the right answer as shown in the blue boxes, but sari answered the question faster than Mohammed so sari will take full point but Mohammed will take a part of point.

The Formula of points by the time for the right answers is :

$$= \frac{1}{\text{Number of players} - \text{Number of player not answered yet}}$$

$$\text{For sari} = \frac{1}{2-1} = 1 \text{ point}$$

$$\text{For mohammed} = \frac{1}{2-0} = 0.5 \text{ point}$$

```
C:\Program Files\WindowsAp x + v
waiting for answer.....
waiting for answer.....
waiting for answer.....
waiting for answer.....
waiting for answer.....
waiting for answer.....
waiting for answer.....
alquds who answer it Mohammed ('192.168.0.189', 60018) Right answer
waiting for answer.....
waiting for answer.....
waiting for answer.....
waiting for answer.....
waiting for answer.....
waiting for answer.....
sari total 1.0 this Question 1.0
Mohammed total 0.5 this Question 0.5

C:\Program Files\WindowsAp x + v
1- What is the capital of Palestine?
enter exit to end the game
please enter your answer:
alquds
enter exit to end the game
please enter your answer:

right answer :alquds
Time up!

SCORE:
sari total 1.0 this Question 1.0
Mohammed total 0.5 this Question 0.5

Command Prompt x + v
C:\Users\hp\Desktop\Task3>client.py

C:\Program Files\WindowsAp x + v
1- What is the capital of Palestine?
enter exit to end the game
please enter your answer:
Alquds
enter exit to end the game
please enter your answer:

right answer :alquds
Time up!

SCORE:
sari total 1.0 this Question 1.0
Mohammed total 0.5 this Question 0.5
```

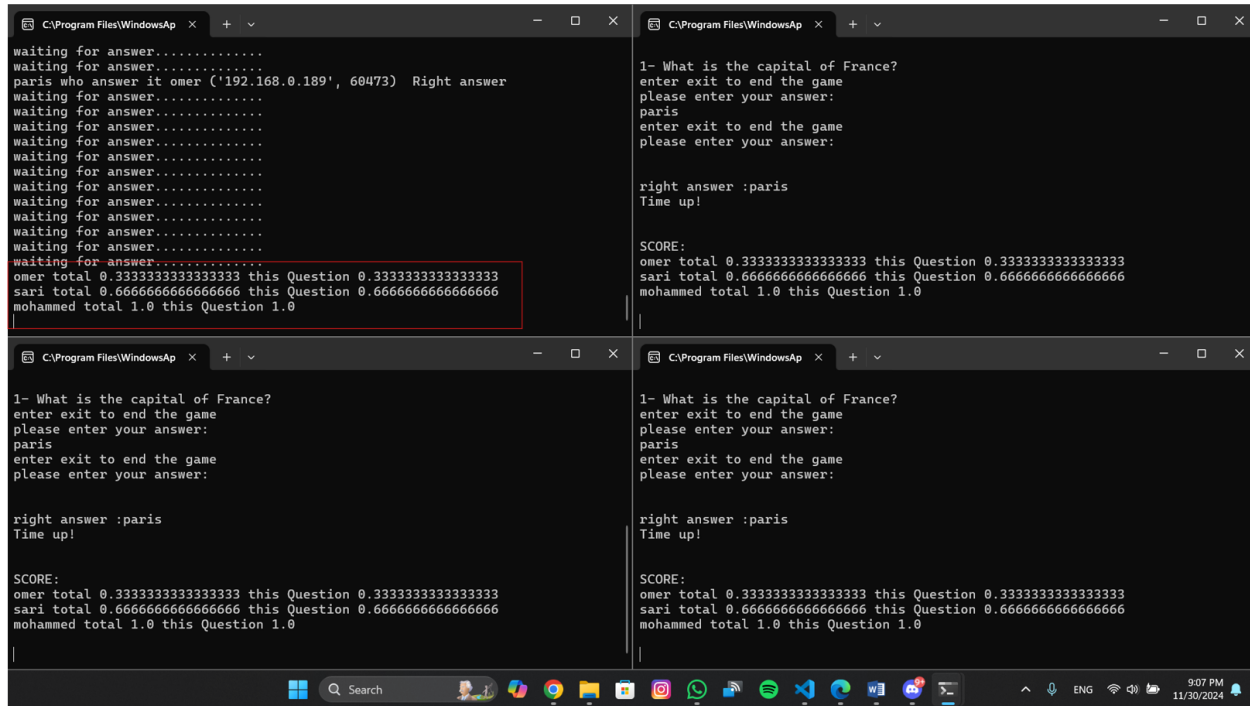
Figure 56: server send the score of the players

Another example if 3 players write the right answer in order to the time:

First one is mohammed take 1 point

2<sup>nd</sup> one is sari take 0.6666 point

3<sup>ed</sup> one is omer take 0.3333 point



```
C:\Program Files\WindowsAp x + v
waiting for answer.....
waiting for answer.....
paris who answer it omer ('192.168.0.189', 60473) Right answer
waiting for answer.....
waiting for answer.....
waiting for answer.....
waiting for answer.....
waiting for answer.....
waiting for answer.....
waiting for answer.....
waiting for answer.....
waiting for answer.....
waiting for answer.....
waiting for answer.....
waiting for answer.....
waiting for answer.....
waiting for answer.....
waiting for answer.....
omer total 0.3333333333333333 this Question 0.3333333333333333
sari total 0.6666666666666666 this Question 0.6666666666666666
mohammed total 1.0 this Question 1.0

C:\Program Files\WindowsAp x + v
1- What is the capital of France?
enter exit to end the game
please enter your answer:
paris
enter exit to end the game
please enter your answer:

right answer :paris
Time up!

SCORE:
omer total 0.3333333333333333 this Question 0.3333333333333333
sari total 0.6666666666666666 this Question 0.6666666666666666
mohammed total 1.0 this Question 1.0

C:\Program Files\WindowsAp x + v
1- What is the capital of France?
enter exit to end the game
please enter your answer:
paris
enter exit to end the game
please enter your answer:

right answer :paris
Time up!

SCORE:
omer total 0.3333333333333333 this Question 0.3333333333333333
sari total 0.6666666666666666 this Question 0.6666666666666666
mohammed total 1.0 this Question 1.0

C:\Program Files\WindowsAp x + v
1- What is the capital of France?
enter exit to end the game
please enter your answer:
paris
enter exit to end the game
please enter your answer:

right answer :paris
Time up!

SCORE:
omer total 0.3333333333333333 this Question 0.3333333333333333
sari total 0.6666666666666666 this Question 0.6666666666666666
mohammed total 1.0 this Question 1.0
```

Figure 57: 3 players with right answers

The following Figure below show the code of receive question and send the question to the server from player:

```

40     elif type == "Q":
41         time.sleep(1)
42         theQuestion = serverMessage.split("@")[1]
43         print("\n")
44         print(theQuestion)
45
46         print("enter exit to end the game\nplease enter your answer:")
47         result = input()
48         exit=True
49         if result == "exit":
50             client.sendto("E@exit".encode(FORMAT), (IPServer, PORT))
51             exit=False
52         else:
53             result = "A@" + str(result)
54             client.sendto(result.encode(FORMAT), (IPServer, PORT))
55         while canNotAnswer:
56             if exit:
57
58                 print("enter exit to end the game\nplease enter your answer:")
59                 result = input()
60                 if result == "exit" and exit:
61                     client.sendto("E@exit".encode(FORMAT), (IPServer, PORT))
62                     exit=False
63                 elif result != "exit":
64                     result = "A@" + str(result)
65                     client.sendto(result.encode(FORMAT), (IPServer, PORT))
66
67         lock.acquire()
68         canNotAnswer = True
69         lock.release()

```

Figure 58: code for Receive the question and answer it

The figures bellow show how the server receive the answer in range of time and check if the answer is right or wrong:

```

219         timer=time.time()
220         server.settimeout(0.01)
221         while (time.time() - timer) < 0.5:
222             try:
223                 message, address = server.recvfrom(1024)
224                 theMessage = message.decode(FORMAT)
225                 type = theMessage.split("@")[0]
226                 if type == "A":
227                     print("duplicate removed")
228                 else:
229                     if address not in activeClients:
230                         server.sendto(questionMessage.encode(FORMAT), address)
231                 thread = threading.Thread(target=handle_client, args=(message, address))
232                 thread.start()
233                 thread.join()
234             except socket.timeout:
235                 pass
236
237
238         server.settimeout(1)
239         startTime = time.time()
240         while (time.time() - startTime) < timeForResponse:
241             try:
242                 message, address = server.recvfrom(1024)
243                 if address not in activeClients:
244                     server.sendto(questionMessage.encode(FORMAT), address)
245                 thread = threading.Thread(target=handle_client, args=(message, address))
246                 thread.start()
247                 thread.join()

```

Figure 59: receive the answer

```

124 elif type=="A":
125     state=checkTheChanceAndUpdate(address)
126     if state == True:
127         theAnswer = clientMessage.split("@")[1]
128         theAnswer = str(theAnswer).lower()
129         whoAnswer=""
130         for client in nameOfClient:
131             if client[1] == address:
132                 whoAnswer=client[0]
133         if theAnswer == theRightAnswer:
134             print(theAnswer, "who answer it", whoAnswer,address," Right answer" )
135         else:
136             print(theAnswer, "who answer it", whoAnswer,address, " Wrong answer")
137         checkTheAnswerAndUpdate(address,theAnswer)
138

```

Figure 60: check player answer

The server received one answer from players, if any player answered many time then the server will only take the first answer, and give the player his point dependent on first answer.

In the figure below show how mohammed (bottom-right) answered 90 which is wrong answer then answered 100 (the right answer) but the server doesn't accept his answer:

The screenshot displays a game interface with four panes. The top-left pane shows server logs with messages like "waiting for answer..." and "100 who answer it sari ('192.168.0.189', 60017) Right answer". The top-right pane shows the game status, including "right answer :alquds", "Time up!", and scores for sari (1.0) and Mohammed (0.5). The bottom-left pane is a Command Prompt showing the command "C:\Users\hp\Desktop\Task3>cclient.py". The bottom-right pane shows a question "2- What is the boiling point of water in Celsius?" with input fields showing "90" and "100".

Figure 61: duplicate answer

If any player was his first answer wrong the player will not take any point, as shown in the below figure mohammed was write duplicate answer, but his first answer was wrong so he wont take any point:

The screenshot displays a game window with a dark background. The text in the window is as follows:

```

waiting for answer.....
waiting for answer.....
waiting for answer.....
waiting for answer.....
waiting for answer.....
waiting for answer.....
waiting for answer.....
waiting for answer.....
waiting for answer.....
waiting for answer.....
waiting for answer.....
sari total 2.0 this Question 1.0
Mohammed total 0.5 this Question 0
  
```

Below this, a red box highlights the following text:

```

2- What is the boiling point of water in Celsius?
enter exit to end the game
please enter your answer:
100
enter exit to end the game
please enter your answer:

right answer :100
Time up!

SCORE:
sari total 2.0 this Question 1.0
Mohammed total 0.5 this Question 0
  
```

The Command Prompt window below shows the command `C:\Users\hp\Desktop\Task3>client.py` being executed.

Figure 62: No point for wrong answer

When the round is end the server send to the client the winner of the round and how many round it wins, in addition to time to start next round:

The screenshot displays a game window with a dark background. The text in the window is as follows:

```

waiting for answer.....
waiting for answer.....
au who answer it Mohammed ('192.168.0.189', 60018) Right answer
waiting for answer.....
waiting for answer.....
waiting for answer.....
waiting for answer.....
waiting for answer.....
waiting for answer.....
waiting for answer.....
waiting for answer.....
waiting for answer.....
sari total 2.0 this Question 0
Mohammed total 1.5 this Question 1.0
the winner this round is: sari with total rounds :1
  
```

Below this, a green box highlights the following text:

```

Next round after: 10
the winner this round is: sari with total rounds :1
  
```

The Command Prompt window below shows the command `C:\Users\hp\Desktop\Task3>client.py` being executed.

Figure 63: winner of the round

The below code show how the server know the winner and send to the clients a message. In addition reset points of all client for the round:

```

86 def findWinnerThisRoundAndPrintIt():
87     if len(nameOfClient)>0:
88         max=nameOfClient[0][3]
89         playerInfo=nameOfClient[0]
90         for client in nameOfClient:
91             if client[3] > max:
92                 max=client[3]
93                 playerInfo=client
94
95     playerInfo[2]+=1
96     winnerMessage="the winner this round is: "+str(playerInfo[0])+" with total rounds :"+str(playerInfo[2])
97     print(winnerMessage)
98     clientWinnerMessage="T@"+winnerMessage
99     for client in activeClients:
100         server.sendto(clientWinnerMessage.encode(FORMAT), client)
101     for client in nameOfClient:
102         client[3] = 0
103

```

Figure 64: server print the winner

If any player wants to join in the game the server will allow to him to join in any time, In the following figure omer ask to join when the game was start but the server accept him directly because our server using threads.

In addition notes that the server send to all players message contain information about omer (IP,name and port):

```

C:\Program Files\WindowsAp x + v
waiting for answer.....
waiting for answer.....
waiting for answer.....
waiting for answer.....
waiting for answer.....
waiting for answer.....
waiting for answer.....
waiting for answer.....
waiting for answer.....
waiting for answer.....
waiting for answer.....
waiting for answer.....
waiting for answer.....
waiting for answer.....
waiting for answer.....
====>omer joined the game, with IP address and Port: ('192.168.0.189', 52149)

C:\Program Files\WindowsAp x + v
Enter your name: omer
please enter the IP: 192.168.0.189
please enter the port: 5689
connected to IP: 192.168.0.189 Port: 5689

C:\Program Files\WindowsAp x + v
Next round after: 10
the winner this round is: sari with total rounds :1

the game start after: 20 time for answer 30

1- What is the square root of 81?
enter exit to end the game
please enter your answer:
8
enter exit to end the game
please enter your answer:
9
omer with address: ('192.168.0.189', 52149) joined the game.....
number of current client : 2

C:\Program Files\WindowsAp x + v
Next round after: 10
the winner this round is: sari with total rounds :1

the game start after: 20 time for answer 30

1- What is the square root of 81?
enter exit to end the game
please enter your answer:
9
enter exit to end the game
please enter your answer:
omer with address: ('192.168.0.189', 52149) joined the game.....
number of current client : 2

```

Figure 65: player join after game start



If any player doesn't answer all question in a round then the server will remove this player from the game and show message of remove.

For example the below figure show when sari (top-right) doesn't write anything in full round the server remove him from the game and continue with other player:

The figure consists of four terminal windows arranged in a 2x2 grid, showing the progression of a game. Each window has a title bar that reads 'C:\Program Files\WindowsAp'.

- Top-left window:** Shows a series of 'waiting for answer.....' messages. It then displays the scores: 'sari total 0.0 this Question 0', 'mohammed total 1.666666666666665 this Question 0', and 'omer total 1.0 this Question 0'. A message 'sari removed.....' is shown, followed by 'the winner this round is: mohammed with total rounds :1'.
- Top-right window:** Shows a question: '3- What is the largest planet in our solar system?'. It prompts 'enter exit to end the game' and 'please enter your answer:'. The 'right answer :jupiter' is shown, followed by 'Time up!'. The scores are then displayed: 'SCORE: sari total 0.0 this Question 0', 'mohammed total 1.666666666666665 this Question 0', and 'omer total 1.0 this Question 0'. A red box highlights the text 'GAME OVER!'.
- Bottom-left window:** Shows the prompt 'please enter your answer:'. The 'right answer :jupiter' is shown, followed by 'Time up!'. The scores are displayed: 'SCORE: sari total 0.0 this Question 0', 'mohammed total 1.666666666666665 this Question 0', and 'omer total 1.0 this Question 0'. The message 'the winner this round is: mohammed with total rounds :1' is shown, followed by 'Next round after: 10'.
- Bottom-right window:** Shows the prompt 'please enter your answer:'. The 'right answer :jupiter' is shown, followed by 'Time up!'. The scores are displayed: 'SCORE: sari total 0.0 this Question 0', 'mohammed total 1.666666666666665 this Question 0', and 'omer total 1.0 this Question 0'. The message 'the winner this round is: mohammed with total rounds :1' is shown, followed by 'Next round after: 10'.

Figure 66: remover player from the game

If any player want to exit from the game then he just write exit the server will remove him from the game.

For example sari write exit then the server send to other players that sari left the game:

Figure 67: exit the game

The client sends to the server message to exit, we insert at the first of message “E@” to make it easier for the server for to read it and handle it:

```

60         if result == "exit" and exit:
61             client.sendto("E@exit".encode(FORMAT), (IPServer, PORT))
62             exit=False
63         elif result != "exit":

```

Figure 68: client send exit to server

The server will receive his message and handle it:

```

139         elif type=="E":
140             whoLeft ="T@left the game:"
141             for client in nameOfClient:
142                 if address == client[1]:
143                     server.sendto("E@GAME OVER!".encode(FORMAT), client[1])
144                     whoLeft += client[0]
145                     nameOfClient.remove(client)
146                     activeClients.remove(address)
147                     if address in answerClient:
148                         answerClient.remove(address)
149
150
151             for client in nameOfClient:
152                 server.sendto(whoLeft.encode(FORMAT), client[1])
153

```

Figure 69: server receive exit message and remove him

### 3.5 using another device:

For our program any player wants to join to the game he can but there are some rules to join:

1. The device should be connected to the same local network that the server on it.
2. The device should have just the file client.py

We try to join from other device then the player was joined and play with another player, but we can't take snapshot from the other device.

The following figures shows that another player with name omer joined to the game and play, but omer has a different IP address:

Sari and server have IP : 192.168.0.189

Omer has IP address: 192.168.0.188

Which mean that omer join from another device in same local network:

The screenshot displays two terminal windows side-by-side, illustrating a game's execution. The left window shows the server's perspective, while the right window shows the client's perspective.

**Left Terminal (Server Side):**

- Initial state: "waiting for response", "active clients: 0". This sequence repeats multiple times.
- A red box highlights the event: "====>sari joined the game, with IP address and Port: ('192.168.0.189', 58603)".
- Following this, it says "waiting for clients to be more than 2" and "active clients: 1".
- Another green box highlights: "====>Omer joined the game, with IP address and Port: ('192.168.0.188', 60875)".
- This is followed by "h2o" and a series of "waiting for answer....." prompts.

**Right Terminal (Client Side):**

- A red box contains the initial input sequence:

```
Enter your name: sari  
please enter the IP: 192.168.0.189  
please enter the port: 5689  
connected to IP: 192.168.0.189 Port: 5689
```
- Below this, it shows "waiting for clients to be more than 2" and "active clients: 1".
- A green box highlights the received message: "Omer with address: ('192.168.0.188', 60875) joined the game.... number of current client :".
- It then displays "the game start after: 20 time for answer 30" and "1".
- The prompt "1- What is the chemical formula for water?" is shown, along with instructions to "enter exit to end the game" and "please enter your answer:". A cursor is visible at the end of the last line.

Figure 70: another device join to the game



## Alternative Solutions, Issues, and Limitations:

1. In task 1 telnet doesn't work on windows, so we try it on Linux operating system.
2. In task 2 we can't open the website from other host until we changed the bind function first parameter to "0.0.0.0".
3. In task 2 the server in our laptop did not response until we turn off the firewall .
4. In task 3 if any player join in the last second in last question and he didn't answer the question the server will kick him.

## References

- [1] <https://www.cb nuggets.com/blog/technology/networking/what-are-transport-layer-protocols>
- [2] <https://www.javatpoint.com/tcp>
- [3] <https://www.geeksforgeeks.org/user-datagram-protocol-udp/>
- [4] <https://www.zenarmor.com/docs/network-basics/what-is-client-server-network>

## Teamwork

We all did the same work at the same time and we have a discord call and work together:

- ⇒ Mohammad Zaid: 33.3%
- ⇒ Mohammad Omar: 33.3%
- ⇒ Sari Abdal-ghani: 33.3%

